Agent Evolution Trajectories: Mechanisms and Pathways in Self-Evolving LLM Agents

Anonymous Author(s)

Surveying the rapidly expanding literature on self-evolving agents, this review makes three contributions: it disentangles the umbrella term "self-evolution" into a precise, operational definition; it organizes the field along the mechanisms that actually drive improvement; and it supplies the measurement vocabulary and threat model that current work lacks. We argue that self-evolution should be reserved for systems in which an agent's own trajectories durably rewrite either the base model's parameters or its external scaffolding, and in which improvement persists across tasks rather than being confined to within-episode retries—a two-criterion boundary that separates genuine evolution from prompt-level self-correction.

Progress has been hampered by conceptual conflation and by evaluation practice that cannot see change. Memory updating and policy optimization are routinely treated as distinct techniques, yet they are two parameterization sites of a single evolutionary loop, differing in update carrier, learning signal, and anti-degradation mechanism; conflating them obscures why externalized memory favors auditability while internalized programs favor transferability. Worse, accuracy, success rate, F1, and pass@1 are static snapshots whose implicit assumption—that the evaluated agent is frozen and each evaluation is independent—is precisely what self-evolution violates, so a single score cannot distinguish growth, stagnation, and erosion.

We therefore synthesize the literature around a carrier-and-persistence framework, tracing a four-stage lineage from implicit ReAct-style reasoning through symbolic learning and skill-library reinforcement learning to protocolized recursive self-evolution, and we map each adaptation operation to its corresponding attack surface. Evidence is drawn from a evidence base of several hundred primary papers, anchored on two frozen works and extended through verifiable 'extends' edges, supplemented by longitudinal benchmarks. The analysis shows that attack surfaces couple to evolution mechanisms operationally rather than modularly: the causal bottleneck is the credit-assignment step that promotes a record to reusable experience, and mechanisms involving abstraction or compression amplify contamination by fixing transient experience into persistent capability that survives its source.

Our strongest conclusion is that the field's missing infrastructure is not greater precision but trajectory-level and incremental metrics—improvement rate, retention, transfer, and forgetting—together with a threat model organized around update operations. Providing both, this review reframes self-evolution as a measurable, auditable process and offers a shared foundation for building agents whose improvement can be demonstrated rather than assumed.

Additional Key Words and Phrases: multi-agent collaboration, large language models, agent orchestration, evaluation benchmarks, agent security

1 INTRODUCTION

Introduction

Large language model (LLM) agents are increasingly deployed as long-running systems that act, observe, and revise their own behavior across many tasks rather than within a single episode. A growing body of work describes such systems as self-evolving: they update reusable state—model parameters, memory entries, tool definitions, skill libraries, or workflows—on the basis of interaction, so that past events become causes of future behavior. Surveys have mapped this space along the axes of what evolves, when evolution occurs, and where the evolved substrate resides. Yet the literature remains organized by mechanism families—reflection, memory augmentation, skill acquisition, prompt and tool optimization—rather than by the temporal object these mechanisms jointly produce. The consequence is a fragmented vocabulary: "self-improvement" denotes anything from re-answering a single question after a reflection step to the persistent rewriting of an agent's symbolic components, and the same evaluation number can stand for growth, stagnation, or decay.

The central tension is that evolution is inherently a trajectory-level phenomenon, while both the methods and the measurements used to study it are episode-level. Methods commit changes at different parameterization sites—external memory written back under a frozen backbone versus internalized prompts, tools, pipelines, or weights —and these sites differ in update signal, auditability, and transfer behavior. Benchmarks, in turn, still report terminal accuracy or pass@1 on frozen task sets, implicitly assuming a stationary agent and independent trials. Longitudinal protocols expose what such snapshots hide: no self-evolution configuration improves monotonically across eight rounds, and none reliably surpasses a non-evolving reference at the end, while multi-stage sequences reveal cross-task interference that single-domain evaluation systematically conceals. Safety analyses sharpen the same point: the attack surface is coupled not to modules but to the adaptation operation that converts experience into a persistent artifact, so that a poisoned trajectory is far more detectable than the skill distilled from it.

This review argues that these disparate findings are observations of one object: the agent's evolution trajectory, defined as the ordered sequence of committed changes to its reusable state, together with the interaction, feedback, and experience that drive them. Adopting this frame yields three shifts relative to existing surveys. First, it replaces mechanism taxonomies with a two-criterion boundary—whether the evolved substrate lies inside or outside the base model, and whether the change is committed across tasks rather than retried within one—so that reflection-only retries and persistent symbolic rewriting are separated by definition rather than by degree. Second, it treats memory update and policy optimization as two parameterization sites of a single loop, distinguished by carrier, update signal, and anti-degradation mechanism, which explains why memory artifacts transfer across backbones while pipeline-coupled policies do not. Third, it reframes evaluation and security as properties of trajectories rather than of endpoints: improvement rate, retention, and transfer replace static snapshots, and persistence, promotion, and attribution replace module-level threat models.

The frame also accommodates the field's developmental structure. The lineage from symbolic learning, which optimizes prompts, tools, and pipelines as a symbolic network under language-based losses, through reinforcement-learned skill libraries, to protocol-level and recursive self-evolution is a sequence of relocations of the evolution carrier, not a sequence of unrelated techniques. Related lines—experience-driven lifelong learning, test-time adaptation, context and memory engineering, and verified synthesis of evolving agents —become comparable positions along the same trajectory rather than separate subfields.

This review makes the following contributions.

- A operational definition of self-evolution. We separate five incommensurable senses of the term using two orthogonal criteria—evolved substrate and cross-task commitment—and show that only trajectory-driven, persistently committed change qualifies, distinguishing it from within-episode retry and from transient working-state accumulation.

- A unified account of memory update and policy optimization. We characterize them as two parameterization sites of one evolution loop, differing in carrier, update signal, and anti-degradation mechanism, and derive the resulting asymmetry in auditability and cross-backbone transfer.

- A critique of static evaluation and a trajectory-level metric agenda. We explain why accuracy, success rate, F1, and pass@1 cannot distinguish growth from stagnation from erosion under self-evolution, and motivate improvement rate, retention, transfer, and forgetting as necessary dimensions.

- A trajectory-coupled threat model. We show that attacks align with adaptation operations rather than modules, that experience-to-skill promotion is the causal bottleneck, and that cross-layer promotion lets transient contamination survive deletion of its source records.

- A reconstructable evolutionary lineage. We trace a four-stage progression—implicit ReAct-style reasoning, symbolic learning over prompts, tools, and pipelines, reinforcement-learned skill libraries, and protocol-level recursive self-evolution —and identify the hubs through which the field radiates rather than a single rooted path.

2 BACKGROUND AND PRELIMINARIES

This section fixes the vocabulary used throughout the review. We do not attempt a complete survey of agent architectures here; instead, we define the small set of constructs that recur in every subsequent theme chapter and state how they relate. The definitions are deliberately operational: each concept is tied to observable artifacts (stored records, updated parameters, executed trajectories, measured scores) so that later claims about evolution, measurement, and safety can be stated precisely.

This four-stage progression reveals how agent capabilities are gradually externalized and accumulated across multiple levels. To consistently track these evolutionary paths across the subsequent topics, a precise and operational terminological framework is first needed to anchor those recurring core components.

2.1 Self-Evolving Agents and Evolutionary Trajectories

A self-evolving agent is an agent whose own artifacts—rather than only its task outputs—are modified as a consequence of experience, such that later behavior differs systematically from earlier behavior on comparable inputs. This definition separates self-evolution from ordinary in-context adaptation: a single prompt that conditions on a retrieved example is adaptation, whereas a persistent change to what the agent stores, retrieves, or executes is evolution. Recent surveys converge on this distinction while differing in scope, with some organizing the field by what evolves, when it evolves, and how the update is driven [1], and others framing self-evolution as the bridge between foundation models and lifelong agentic systems. Work on test-time learning sharpens the temporal boundary further, placing evolution inside deployment rather than in a separate offline training phase [2].

We characterize an agent's evolutionary trajectory as the ordered sequence of artifact states the agent passes through, together with the evidence that produced each transition. This notion is distinct from a task trajectory (a sequence of actions within one episode): a task trajectory is the unit of behavior, whereas an evolutionary trajectory is the unit of change across episodes, sessions, or tasks. The distinction matters because the two are frequently conflated in evaluation, where a strong task trajectory is read as evidence of evolution even when no artifact has durably changed. Frameworks that learn on the job make the separation explicit by recording which experiences were retained and how they altered subsequent behavior, and protocol-level proposals attempt to standardize the transition itself so that evolutionary trajectories become comparable across systems.

The trajectory framing also supplies the review's central organizing claim: heterogeneous mechanisms can be compared by where on the trajectory they act and what they carry forward. An agent that consolidates successful episodes into reusable memory and an agent that rewrites its own tool-use policy occupy different positions on the same trajectory even when both are described as "self-improving". Making this position explicit is what allows the five mechanism families discussed later to be treated as comparable rather than merely co-existing.

2.2 Evolutionary Carriers: Memory, Policy, Skills, and Workflows

An evolutionary carrier is the substrate that persists across episodes and is modified by evolution. We distinguish four carriers that recur in the literature. Memory comprises stored experience records, episodic traces, and their retrieval structures [3]; hierarchical and generative variants differ in how records are organized and reconstructed but share the property that retrieval, not parameter change, is the locus of adaptation. Policy denotes the mapping from observations to actions, whether realized as model parameters, prompts, or a controller; policy-level evolution changes this mapping through optimization or search. Skill libraries are collections of reusable, named competencies with applicability conditions, and workflows are reusable multi-step procedures, often expressed as graphs or protocols.

The carriers are not independent. A skill is typically distilled from memory records of successful episodes, and a workflow is a composition of skills with control flow; consequently, evolution at one carrier can propagate to others. This coupling is why we treat carriers as a set rather than a hierarchy: a system may evolve memory while leaving policy fixed, or evolve policy while memory remains a passive log, and the two choices yield different capability profiles and different failure modes. The externalization perspective is useful here because it treats memory, skills, and protocols as artifacts that can be inspected, versioned, and audited independently of the model weights that consume them [3].

Two clarifications prevent later confusion. First, a carrier is defined by persistence, not by representation: a prompt template that is rewritten and reused is a policy carrier, while a retrieved document that is never written back is not a carrier at all. Second, carrier identity is orthogonal to mechanism: the same memory substrate can be updated by reinforcement learning over episodic records [4] or by reflective consolidation [5, 6], and these are different evolutionary mechanisms acting on the same carrier.

2.3 Experience Storage versus Policy Optimization

The most consequential distinction in this review is between the experience storage layer and the policy update layer. The storage layer governs what is written, retained, and retrieved: record selection, deduplication, abstraction, and forgetting. The policy layer governs how the agent's action-selection mapping changes: gradient updates, prompt or program search, and tool-policy revision [2]. The two layers interact—stored experience supplies the signal for policy updates, and policy determines which experiences are generated—but they are analytically separable, and conflating them is a common source of overclaimed

results.

The distinction has direct methodological consequences. Storage-layer methods are evaluated by retrieval quality and downstream task gain under a fixed policy, and their characteristic failure is memory misevolution, where poisoned or stale records degrade behavior without any change to the model. Policy-layer methods are evaluated by improvement in the action mapping itself, and their characteristic failure is overfitting to a narrow experience distribution. Because the two failures require different diagnostics, evaluation protocols that report a single aggregate score cannot distinguish them; this motivates the multidimensional measurement discussed in the next subsection.

A second consequence concerns attribution. When a system improves after accumulating experience, the improvement may originate in better retrieval, a genuinely better policy, or an interaction between the two. Designs that make the storage layer explicit and inspectable—experience graphs, curated skill memories, consolidated records—allow this attribution to be performed. We therefore treat the storage/policy split not as a taxonomy of methods but as an accounting principle that later chapters apply when comparing mechanism families.

2.4 Capability Measurement and Evaluation Protocols

We define capability as the expected task performance of an agent under a specified distribution of tasks and a specified evaluation protocol, and a static capability snapshot as a capability estimate computed from a single evaluation episode or a single fixed checkpoint. Conventional metrics—accuracy, success rate, F1, and pass@k—are snapshots in this sense: they summarize behavior at one point on the evolutionary trajectory and are silent about retention, transfer, and degradation [7]. This is not a defect of the metrics themselves but a mismatch between what they measure and what evolution claims to produce.

A longitudinal evaluation protocol measures capability as a function of position on the trajectory, repeating assessment across sessions and reporting retention and forgetting alongside acquisition. Benchmarks designed for lifelong and long-horizon settings operationalize parts of this protocol by interleaving tasks and probing earlier competencies after later learning [7], and test-time scaling studies show that gains observed at a single budget can reverse when the budget or the task horizon changes. Evolutionary test-time learning frameworks extend the protocol to the adaptation process itself, evaluating whether the trajectory converges rather than whether a single endpoint is strong.

Measurement also determines what counts as an evolutionary signal. In closed tasks with reliable verifiers, feedback is a low-noise signal; in open tasks, the verifier is itself a model or a heuristic, and the signal can be corrupted by the evaluator's errors. When this happens, optimization against the signal produces improvement on the signal without improvement in capability—evolution degenerates into fitting evaluator noise. Verified synthesis and co-evolutionary verification are two responses to this problem, both of which make the verification step an explicit, auditable component of the evolutionary loop. We treat verifier reliability, not task difficulty, as the primary axis along which evaluation protocols for self-evolving agents should be compared.

2.5 Safety Constraints and the Attack Surface of Evolution

A safety constraint is a requirement that the agent's behavior remain within specified bounds across its entire evolutionary trajectory, not merely at deployment time. This framing follows from the observation that evolution enlarges the attack surface: every carrier that can be updated is a channel through which an adversary can influence future behavior [8]. The coupling is mechanism-specific. Poisoned experience records target the storage layer and can persist long after the inducing input is gone [9]; compromised tool feedback targets the policy layer by corrupting the signal used for revision [10]; and termination or control-flow manipulation targets workflows and protocols. Because these attacks act on different carriers, defenses designed for one channel do not transfer to another.

The literature has begun to name this class of risk explicitly. Memory misevolution benchmarks measure degradation caused by adversarial or merely erroneous records [11], and analyses of experience-driven agents identify the retention of harmful experience as a distinct threat category [9]. Persistent-control studies show that an adversary who influences an early update can retain influence over later behavior, which is precisely the trajectory-level property that static safety evaluation fails to capture [8]. Integrity-oriented proposals argue that security properties must be enforced on the intent-to-execution path rather than on individual outputs [12], and information-flow analyses provide the tracking machinery needed to attribute a behavioral change to a specific influence [13].

These results motivate the review's treatment of safety as a structural tension rather than an add-

on. Capability evolution requires accepting external signals and retaining them; safety requires bounding which signals are accepted and how retained influence is constrained. The tension is not resolvable by a single mechanism, and the frameworks that address it most directly make the evolutionary loop's components—signal source, carrier, verifier, and update rule—individually inspectable [14, 15]. This is the same accounting principle introduced for the storage/policy split, applied now to adversarial rather than benign influence, and it is the sense in which the concepts defined in this section are meant to be used: as a shared vocabulary for stating precisely where an evolutionary claim, a measurement, or a risk is located on the trajectory.

3 RELATED WORK

3.1 Related Work

Research on LLM-based agents has expanded rapidly, and the survey literature has begun to consolidate this growth along several distinct axes. One strand organizes the field around planning, treating the agent as a controller that decomposes goals into executable steps and reasoning over long horizons [16]. A second strand takes memory as its organizing lens, arguing that long-term retention and retrieval constitute the principal bottleneck for agents that must operate across many sessions [17]. A third strand foregrounds safety, examining how the accumulation of state transforms an agent's threat surface over time [18]. More recent syntheses have attempted a broader framing, either by casting self-evolving agents as dynamic graph transformations or by asking what, when, how, and where an agent should evolve. Taken together, these surveys establish a fragmented field in which each contribution illuminates one mechanism while leaving the relationships among mechanisms largely unexamined.

The symbolic-learning line of work offers the most explicit formalization of self-evolution. It reframes prompts, tools, and control flow as components of a symbolic network and defines language-based loss, gradient, and optimizer operations over that network, so that the agent improves by editing its own symbolic structure rather than by updating model weights. This formulation has the virtue of interpretability: every update is a discrete, inspectable edit. Its limitation is equally clear, since the abstraction presupposes that experience can be verbalized and that the resulting edits transfer across tasks. Our review treats this line as the canonical account of strategy-level evolution and uses it as one anchor for comparing how different mechanisms represent and apply update signals.

A parallel line pursues reinforcement learning over skill libraries. Here the agent accumulates reusable routines and learns, through reward-driven optimization, when to invoke and how to refine them. Related work on trajectory optimization for LLM agents shows that exploration-based trial and error can yield measurable improvement without parameter updates, while contrastive reasoning over tool-use traces improves tool selection. These studies share a commitment to treating the skill library as the evolving substrate, but they differ sharply in what counts as an update signal—scalar reward in one case, comparative preference in another. We build on this line while arguing that the choice of substrate, not only the choice of optimizer, determines what kinds of capability change are possible.

Memory-centered approaches externalize the mutable state of an agent into retrievable stores. Retrieval-augmented generation provides the foundational mechanism, and subsequent work has refined it into structured forms: workflow memories that record reusable action sequences, autonomous memory augmentation that decides what to retain [19], and runtime reinforcement learning over intent–experience–utility triples. Empirical studies of very long-term conversational memory demonstrate both the promise and the fragility of these designs, and position papers argue that episodic rather than semantic memory is the missing ingredient for sustained operation [17]. Our work differs in scope: rather than proposing another memory architecture, we ask how memory updates interact with strategy updates and where the boundary between the two becomes ambiguous.

Reflection and self-correction constitute a third mechanism family. Controlled studies of self-reflection show that prompting an agent to critique its own prior attempts improves problem-solving performance, though the gains depend strongly on task structure [20]. Integrated systems combine reflection with memory augmentation so that lessons drawn from failure persist across episodes, and self-taught action deliberation extends this idea by having agents compare candidate actions before committing. Human-in-the-loop variants further allow test-time learning when environment rules shift [21]. We treat reflection as the mechanism that generates update signals rather than as an end in itself, which lets us compare it directly against reward-based and memory-based routes.

Workflow-level adaptation has emerged as a distinct route in which the unit of evolution is the control flow itself. Agent workflow memory induces reusable workflows from successful trajectories and injects them into later tasks, and related work on interactive workflow provenance provides reference architectures for

tracking how such workflows change. Trajectory reduction techniques lower the cost of long interaction histories and thereby make workflow-level accumulation practical [22]. At the systems level, researchers have drawn explicit analogies between agent construction and computer systems design [23], and zero-code frameworks lower the barrier to composing such workflows. This literature is rich in engineering insight but thin on comparative analysis, and our review addresses that gap by placing workflow adaptation on the same axis as memory and strategy evolution.

Multi-agent collaboration extends evolution from the individual to the collective. Studies of social-norm evolution in LLM populations show that conventions emerge and stabilize through natural-language interaction [24], and multi-agent systems for strategic planning demonstrate continual learning over long horizons in adversarial settings. Frameworks for automated research coordinate specialized agents toward shared goals [25], while moral-alignment work examines how normative constraints propagate across agent collectives. These studies raise a question that single-agent accounts cannot pose: whether the evolving unit is the individual or the interaction pattern. We incorporate this perspective as a fifth mechanism family and argue that collective evolution exhibits distinct convergence properties.

Tool use forms another well-developed strand, in which the agent's evolving capability resides in the tools it can construct, select, and compose. Methods have been proposed for having agents author their own tools [26], for optimizing tool usage through contrastive reasoning, and for harvesting tools across pools in pooled-tool environments. Domain deployments illustrate the breadth of this route, from metasurface inverse design [27] and building-information modeling to fire dynamics simulation [28] and hyperparameter optimization. Because tool definitions are persistent artifacts, this route raises security questions that are qualitatively different from those of prompt-level adaptation, a point we develop below.

Evaluation of evolving agents has only recently begun to receive sustained attention. Lifelong learning benchmarks test whether agents improve across sequential tasks, and asynchronous planning benchmarks probe whether agents can reason over concurrent state. Event-forecasting evaluations measure an agent's ability to gather and exploit world information over time [29], and live-website A/B testing frameworks evaluate interactive agents in deployment conditions. Yet the dominant metrics remain static task scores such as accuracy and pass rates, which presuppose that the evaluated system does not change during evaluation. We argue that this presupposition is precisely what self-evolution violates, and that capability change requires its own measurement apparatus.

Safety and security research on agents has proceeded largely independently of the evolution literature. Surveys document how accumulated state alters an agent's safety semantics [18], and empirical work shows that commercial agents are vulnerable to simple yet dangerous attacks. Privacy risks in agent memory have been characterized [30], personalized safety benchmarks for long-horizon agents have been introduced, and harmfulness benchmarks measure agent misuse. Mobile-agent security analyses and calls to adopt classical security principles round out this strand [31]. Crucially, this literature rarely asks which evolution mechanisms amplify which attack surfaces, a coupling we treat explicitly.

A growing body of work examines the economics and usability of agentic systems, which bears indirectly on evolution. Analyses of agentic return on investment question whether the cost of long interaction histories is justified, and audits of financial benchmarks show that standard evaluations fail to capture risk-relevant behavior [32]. Cost-reduction techniques for trajectories make persistent evolution more tractable [22]. These contributions remind us that evolution is not free: every retained memory and every refined skill imposes storage, latency, and verification costs that shape which mechanisms are deployable in practice.

Domain applications provide the empirical testing ground for all of the above. Self-improving frameworks have been built for visual question answering and grounded dataset synthesis [33, 34], for sixth-generation network orchestration [35], and for carbon-neutral energy systems. Metacognitive agents that predict their own learning progress offer a mechanism for prioritizing goals in open-ended spaces [36], and constraint-informed learning provides a template for warm-starting optimization from prior experience. These deployments demonstrate that self-evolution is not confined to a single benchmark family, but they also show that evaluation practices remain domain-specific and largely non-comparable.

Finally, a set of foundational and methodological works supplies the conceptual vocabulary that the field currently lacks. Classical results on selection and specialization [37], on algorithm selection and configuration, and on inferring model accuracy from architectural characterizations anticipate questions that agent research is only now confronting: how to allocate evaluation effort, how to compare systems whose behavior changes, and how to predict performance without exhaustive testing. Work on software evolution and execution-log analysis offers methodological precedent for studying change over time [38]. We draw on these ideas to motivate a trajectory-level rather than snapshot-level evaluative stance.

Across these strands, three blind spots recur. First, no existing synthesis places memory update, strategy optimization, tool use, workflow adaptation, and multi-agent collaboration on a single comparative axis; each survey covers one mechanism and leaves cross-mechanism comparison to future work. Second, the evaluation literature has not developed metrics for capability change, so that a single accuracy figure can correspond to growth, stagnation, or erosion of competence. Third, safety research and evolution research remain separate conversations, leaving open which mechanisms enlarge the attack surface and which constraints keep evolution bounded. This review addresses all three by organizing existing work around the notion of an evolution trajectory, comparing mechanisms by their evolutionary substrate, update signal, and convergence behavior, and articulating an evaluation agenda for measuring how agents change.

- [18] — surveys safety in self-evolving agents, establishing that accumulated state changes safety semantics over time.
- [39] — reframes self-evolving agents as dynamic graph transformations and offers a unifying perspective.
- [40] — formalizes prompts, tools, and pipelines as a symbolic network with language-based loss and optimization.
- [41] — learns when to invoke and how to refine a skill library through reinforcement learning.
- [42] — analyzes structural memory as a persistent substrate for long-horizon agent behavior.
- [43] — argues that long-term memory is the principal bottleneck for LLM agents.
- — benchmarks agents as lifelong learners across sequential tasks.
- [44] — demonstrates that deployed commercial agents remain vulnerable to simple attacks.

4 SURVEY METHODOLOGY

Of the 274 screened publications, 88 are cited in the synthesis; the remaining 186 informed screening and background and are not individually cited.

Data Mapping. The review draws on a corpus of 274 publications assembled through multi-source retrieval, with arXiv prioritized as the primary channel and open scholarly indexes used as a fallback to extend coverage. Records were deduplicated by anchoring each entry to a persistent identifier, so that the same work indexed under different metadata could not enter the corpus twice. The scope is defined by five research questions, decomposed into 13 sub-questions that jointly bound what the review is expected to answer. This decomposition was fixed before retrieval was closed, which means the corpus is bounded by the questions rather than by whatever the searches happened to return. Each sub-question therefore carries an explicit remit, and the 274 publications are distributed across these remits rather than pooled into a single undifferentiated set.

After clarifying the stage characteristics and terminology system of agent capability evolution, how the literature evidence supporting these judgments is systematically collected, screened, and organized becomes a key step in ensuring that the survey's conclusions are traceable and reproducible.

Data Refinement. Screening proceeded against pre-specified inclusion criteria tied to the 13 sub-questions, and the resulting evidence set was frozen prior to analysis. This ordering matters: no paper was admitted or excluded on the basis of what the analysis subsequently showed, so the evidence base cannot drift toward findings that are easier to support. For every retained paper, six evidence fields were coded: the problem addressed, the method employed, the datasets and benchmarks used, the metrics reported, the limitations acknowledged, and the assumptions and constraints under which the results hold. The last two fields are the ones most often left implicit in primary papers, and coding them explicitly forces each result to be read together with the conditions under which it was obtained. Coding also produced cross-paper correspondence links, recording where two or more papers address the same problem, share a benchmark, or report on comparable metrics, so that later comparison operates on aligned records rather than on surface similarity of topic.

Data Evaluation. Evaluation proceeds in three layers. The first constructs an evidence matrix in which the 274 papers are indexed against the 13 sub-questions and the six coded fields, making visible both the density of evidence per sub-question and its gaps. The second layer analyses each sub-question in turn along three axes: comparison across papers sharing a problem or benchmark, evolution of method

and reported performance over time, and explicit trade-offs among accuracy, cost, data requirements, and generality. The third layer decomposes the resulting conclusions into individually checkable claims, each annotated with the specific papers that support it. A claim without an identifiable supporting set is not carried forward, and where the evidence matrix shows a sub-question supported by only a thin slice of the corpus, that thinness is stated rather than smoothed over. The output is thus a set of traceable claims whose warrant can be inspected paper by paper.

5 AN EVOLUTIONARY TRAJECTORY FRAMEWORK UNIFORMLY CHARACTERIZES AGENT EVOLUTION

Once the verifiable, paper-by-paper evidence-support relationships have been established, the elaboration of each theme acquires a traceable basis; the following turns to the first thematic dimension, examining the core concerns and evidence distribution of agent research along this dimension.

Having established the methodological apparatus, this chapter turns to the conceptual core of the review: whether "evolution" and "self-improvement" in agent systems name a single phenomenon or a family of loosely related ones. We argue that the term functions as an umbrella shared by at least five incommensurable meanings, and that its operational boundary is best drawn by two orthogonal criteria—the carrier that evolution rewrites (base-model parameters versus extra-model scaffolding and persistent state) and whether improvement constitutes a durable cross-task commitment rather than within-task retry. The analysis proceeds from this definitional tension to a three-layer decomposition, showing that the carrier and update-operator axes form a genuinely orthogonal and MECE skeleton while the update signal belongs to the operator level as a triggering input rather than a third independent dimension, and it closes by sharpening the distinction between memory update and policy optimization along the content-versus-control and externalized-versus-internalized axes—a distinction whose consequences for reproducibility lead directly into the five categories of evolutionary mechanisms examined next.

5.1 Conceptual Boundaries of Evolution in Agentic Systems

What does "evolution" denote when it is applied to an agent, and where should its conceptual boundary be drawn? The term is used across the surveyed literature to cover a spectrum of mechanisms whose only common denominator is that some aspect of the system changes over time, and the analytical task of this subsection is to separate genuine self-evolution from neighbouring phenomena that share its vocabulary but not its commitments. A useful organizing distinction emerges from the literature itself: evolution in the strong sense requires the agent to commit a persistent, reusable change to its own state—parameters, memory, tool definitions, skill libraries, or workflows—so that past interactions become causes of future behaviour. Weaker usages denote transient adaptation within a single episode, and still weaker usages in adjacent fields denote the temporal evolution of a model's outputs rather than of the agent itself. The boundary question is therefore not terminological pedantry; it determines which systems can meaningfully be compared, and which safety and evaluation claims transfer between them.

The strongest available formalization treats self-improvement as a self-induced operator in which the agent at step $t + 1$ is produced by an update function U applied to the interaction history and to an evaluation signal over the policy. The decisive criterion in this formulation is that U must commit a persistent change; this explicitly contrasts with the instantaneous evolution of a context variable X_t that accumulates dialogue history or working state without altering anything the agent will carry forward. This criterion is analytically attractive because it is operational: one can ask of any proposed self-evolving system whether an update survives the episode in which it was produced. It also explains why the same mechanism can be classified differently depending on implementation—a reflection step that only conditions the next answer is not evolution, whereas the same reflection step, when its output is written into a retrievable store, crosses the boundary. The remainder of this subsection applies this criterion to the mechanisms reported in the literature and examines where it strains.

At the weakest end of the spectrum sit single-episode self-correction mechanisms. A systematic study of self-reflection across nine models and eight reflection types found that re-answering a question after reflection improved performance significantly ($p < 0.001$), yet the authors themselves characterize the gain as solving single-step problems rather than building any cross-task capability. Under the persistence criterion this is adaptation, not evolution: nothing is retained once the answer is emitted. The distinction matters because such results are frequently cited as evidence that agents "improve themselves," when in fact they demonstrate only that a second attempt conditioned on a critique can be better than the first. The comparison with cross-task mechanisms is instructive precisely because the surface behaviour—generate, critique, regenerate—is nearly identical, while the persistence properties are opposite.

Cross-task evolution appears when experience is distilled into a form that is retrieved for later tasks. Experiential reflective learning illustrates the transition: injecting retrieved heuristics into the agent raised performance on Gaia2 by 7.8% over a ReAct baseline, and LLM-based retrieval at $k = 20$ reached 56.1% against a 48.3% baseline, while retrieval over raw trajectories actually degraded performance by 1.9% relative to heuristics [6]. This last comparison is the conceptually important one: persistence alone is insufficient, since storing raw experience can transfer noise as readily as signal, and the benefit depends on the abstraction applied before storage. Evolution in this sense is therefore not merely the accumulation of state but the accumulation of useful state, a condition that later subsections will show is difficult to maintain.

A parallel line of work establishes that persistence does not require parameter updates, which further sharpens the boundary. MetaAgent reports self-evolution through meta tool learning with neither parameter updates nor post-training, raising QwQ-32B from 22.3 to 47.6 and Gemini-2.5-Flash from 7.8 to 49.5 on GAIA [10]. SiDiffAgent similarly demonstrates a training-free closed loop in which VQAScore rises from a Qwen-Agents baseline of 0.789 to 0.852, and to 0.884 in a second round that reuses the first round's experience library. Read together, these results indicate that the persistence criterion, rather than the locus of update, is what separates evolution from mere inference-time adaptation; the update may live in a tool library or an experience store rather than in weights. This has a direct consequence for how the field should be surveyed: taxonomies organized by what is updated risk conflating mechanisms that differ in whether anything is updated at all.

The boundary becomes harder to police once the term is followed into adjacent disciplines. In time-series event prediction, "evolution" refers to the drift of a predictive distribution over continuous time [45], and in digital-twin work it refers to versioned updating of a virtual model across a system's lifecycle. Neither usage involves an agent whose own trajectory drives a persistent update to its future behaviour. These are legitimate senses of the word, but importing them into agent research would make the concept coextensive with any temporally varying system and thus unusable as an explanatory category. The contrast is worth stating explicitly because cross-disciplinary retrieval increasingly surfaces such papers alongside agentic ones, and a survey that does not mark the distinction will propagate the ambiguity it is meant to resolve.

Even within agent research, the terminology remains unsettled. Recent surveys observe that closely related mechanisms—self-correction, meta-prompting, self-play—appear under different labels that obscure their underlying similarity, and that the field's conceptual boundaries are still being negotiated [46]. The same fragmentation is visible from the safety side, where the operative definition is framed in terms of reusable state precisely because the safety implications depend on it: once experience becomes reusable state, past events become causes of future behaviour [18]. The two surveys converge on the same underlying criterion while organizing around different concerns, which suggests the criterion is robust to framing. What remains absent is a shared vocabulary for the intermediate cases—systems that persist some state but not others, or that persist it only within a session.

Synthesizing across these positions, three nested senses of evolution can be distinguished, and they are not interchangeable. Transient adaptation covers within-episode revision with no retained state, exemplified by single-pass self-reflection [20]. Persistent self-evolution covers committed updates to reusable state, whether in memory, tools, or skills, and is the sense that both major surveys operationalize. Temporal evolution, finally, covers the drift of a model or distribution over time without any agent-driven update, as in the time-series and digital-twin usages. Only the middle sense supports claims about an agent improving itself, and only that sense carries the safety and evaluation consequences that motivate the rest of this review.

Two caveats qualify this taxonomy. First, the persistence criterion is a necessary rather than a sufficient condition: ERL's finding that raw-trajectory retrieval underperforms heuristics shows that committed state can be actively harmful, so persistence must be paired with a quality condition on what is stored. Second, the boundary between transient and persistent is implementation-dependent rather than architectural, since the same reflection module can fall on either side depending on whether its output is written to a store. These caveats do not undermine the distinction, but they mean that classifying a system requires inspecting its update path rather than its component names—a methodological point that carries into the trajectory framework developed in the following sections.

5.2 Decomposing Evolutionary Trajectories into Carrier, Signal, and Operator Layers

Can an evolutionary trajectory be decomposed into a three-layer structure of evolutionary carriers (memory, policy, skill library, workflow, protocol), update signals (feedback, verifier, reflection), and update operators (write, prune, optimize, compose)? The evidence assembled here supports such a decomposition

only with two qualifications: the carrier axis is robust and well corroborated, whereas the signal axis is better understood as an input to the operator layer than as an independent dimension orthogonal to it.

The carrier dimension receives convergent support from at least four independent lines of work, which is unusual for a construct introduced at this level of abstraction. A safety-oriented survey organizes self-evolving mechanisms into six families that map closely onto distinct substrate types [18], while a comparative reformulation types the nodes of a dynamic graph so that memory, tools, and prompts become formally distinguishable entities. A protocol-level account separates a resource layer comprising prompts, tools, and memory from the evolutionary operations acting upon it [15], and streaming-task evaluations partition agent state into context, memory, skill, and harness components. These partitions are derived from different motivations—safety analysis, formal modeling, interoperability, and benchmarking—yet they converge on a small set of persistent substrates, which indicates that the proposed memory/policy/skill/workflow/protocol taxonomy is a coarse-graining of a real structural regularity rather than an artifact of any single framing.

The operator dimension is likewise defensible as a coarse grouping, although the correspondence is looser than for carriers. Formal treatments enumerate primitive transformations such as node insertion, deletion, feature update, and merging, alongside link creation, removal, and rewiring [39], and safety-oriented analyses add operations for abstraction, modification, and forgetting [18]. The four proposed operators—write, prune, optimize, and compose—can be read as equivalence classes over these primitives: insertion and feature update collapse into write, deletion and forgetting into prune, rewiring and modification into optimize, and merging into compose. This reduction is legitimate at the granularity of trajectory description, but it is not exhaustive. Read-only activation, in which a stored capability is retrieved and applied without altering the store, and migration or propagation, in which knowledge moves between agents or scopes, both appear in the literature yet fall outside the four-class scheme. A trajectory framework that omits them may misclassify retrieval-heavy or multi-agent evolution as the absence of evolution.

A more consequential finding concerns the status of the signal layer. The three-layer picture implicitly treats signals as a dimension orthogonal to carriers and operators, but the surveyed formalisms place signals inside the update operator itself. One survey writes the update explicitly as a function whose arguments include an update signal alongside the current state [47], and the safety survey positions selection and evaluation as stages that precede an operator's commit step rather than as a parallel axis [18]. On this reading, a signal is the input that determines which operator fires and how strongly, not a separate coordinate of the trajectory. The practical implication is that the three-layer decomposition should be understood as two structural axes plus a conditioning input, and that any trajectory schema treating signals as freely combinable with operators will overgenerate configurations that no system instantiates.

The internal partition of signals into feedback, verifier, and reflection is best interpreted as a typology of signal sources rather than a set of mutually exclusive categories, because a single update event can be conditioned on several sources simultaneously. Reflection has been shown to improve problem-solving when injected into the prompt as a self-critique step [20]; a verifier can trigger re-sampling when distributional grounding fails [48]; and an evaluator can rewrite a skill document directly. In each case the signal differs in origin—self-generated critique, external check, or task-level score—yet all three act on the same update event from different sides. Consequently, the signal layer should be recorded as a set of active sources per event, not as a single label, and comparative claims about which signal type is superior are only meaningful when the carrier and operator are held fixed.

Holding the carrier and operator fixed while varying the signal is precisely what the strongest quantitative studies do, and their results illustrate the decomposition's explanatory value. On a skill carrier, a deterministic physics-based evaluator paired with an optimize operator that rewrites a skill document raises success from 38% to 74%, lifts the composite performance factor from 0.51 to 0.87, and reduces mean attempts from 4.10 to 2.30, all while the base model and solver remain fixed [27]. Because only the signal-and-operator pair changes, the gain is attributable to the update mechanism rather than to model capacity—an attribution that would be impossible without the layered decomposition.

A parallel contrast appears when reinforcement signals replace evaluator signals on a skill-library carrier. Driving skill generation with reward and verifier signals yields 72.0% task-goal completion and 60.7% scenario-goal completion on AppWorld, exceeding a pure GRPO baseline at 69.2% and 51.8%, while cutting steps from 16.4 to 12.1 and token consumption from 3,613 to 1,475. The two studies differ in carrier granularity (single document versus library), signal type (deterministic evaluator versus learned reward), and operator (targeted rewrite versus generation), yet both show that pairing an appropriate signal with a matching operator dominates signal-free iteration. The convergence across such different regimes is stronger evidence for the decomposition than either result alone.

Memory carriers exhibit the same pattern with a different operator mix, and here the prune operator

becomes visible in a way the skill studies do not capture. Combining reflection-driven writes with decay and pruning for memory management produces large ablation gains on AgentBench, lifting a small model from 6.8 to 45.3 on knowledge-graph tasks and a code model from 0.0 to 48.0 [49]. The magnitude of the second gain is instructive: a base score of zero indicates that the carrier, not the signal, was the binding constraint, and no amount of reflection would have helped without a store to write into. This is a direct argument for treating carriers as a first-class axis rather than as an implementation detail of the signal.

Policy and reasoning carriers complete the picture, with verifier signals triggering repair operators on intermediate reasoning states. A distributionally grounded agent raises faithfulness and interpretability scores from 51.2 and 44.1 under ReAct to 94.7 and 88.9, and reaches 90.1 localization accuracy and 86.2 factual accuracy [48]. The mechanism differs from the skill and memory cases: the update targets a reasoning trajectory rather than a stored artifact, so the operator is a local repair rather than a global rewrite. That the same signal-operator logic applies across such different substrates supports the generality of the decomposition while cautioning that operator semantics are carrier-dependent.

Two boundaries qualify the framework. First, the operator inventory is incomplete in ways that matter for retrieval-heavy and multi-agent systems, as the omission of read-only activation and migration suggests. Second, the signal typology is not a partition, so trajectory records that assign a single signal label will lose information about compound update events. Neither boundary invalidates the decomposition; both indicate that it should be applied as an analytical lens with explicit scope conditions rather than as an exhaustive ontology.

Taken together, the evidence supports a qualified affirmative: evolutionary trajectories can be decomposed into carrier, signal, and operator layers, provided that signals are modeled as conditioning inputs to operators rather than as an orthogonal axis, and provided that the operator set is extended beyond write, prune, optimize, and compose to cover activation and propagation. This qualified structure supplies the vocabulary needed to compare trajectories across systems without conflating substrate changes with mechanism changes, and it sets up the question of how such trajectories can be compared quantitatively when carriers, signals, and operators vary simultaneously.

5.3 Inter-layer Coupling Between Experience Storage and Policy Updates and Its Reproducibility Cost

How should the experiential storage layer be separated from the policy optimization layer, and what reproducibility failures arise when the boundary between them blurs? The preceding structural vocabulary treats memory and policy as distinct carriers of change, yet the empirical record shows that most self-evolving systems modify both within a single update step, and that this operational coupling—rather than any conceptual ambiguity—is what produces the characteristic failure modes of agent amnesia and skill forgetting.

A first observation is that the nominal separation of layers is widely asserted but rarely enforced in implementation. Several frameworks describe memory-based evolution as leaving the core policy prompt and decision logic untouched, admitting only accumulated experience into the context; in practice, however, the memory update and the guiding prompt are rewritten in one joint operation, so that a change attributed to experience storage simultaneously alters the decision procedure [50]. This pattern of nominal layering with operational coupling is not incidental: it reflects the convenience of a single generative edit over two separately governed artifacts. The consequence is that ablation studies attributing gains to "memory" cannot, under such designs, isolate the memory contribution from a concurrent policy perturbation.

Evidence that the two layers make separable and non-substitutable contributions comes from controlled comparisons on the J-TTL benchmark. Memory-only methods perform markedly worse than prompt-level or whole-system evolution, with memory-only variants reaching approximately 0.13–0.14 and retrieval-augmented variants 0.18–0.19, against 0.31–0.36 for prompt optimization and 0.47–0.50 for full evolutionary test-time learning [50]. The ordering indicates that stored experience alone carries limited task-solving capacity, and that its value is realized only when a policy layer consumes it. A natural inference is that the two layers are complementary rather than redundant, which in turn implies that systems collapsing them into one edit forfeit the ability to tune their relative contributions—precisely the knob that reproducibility requires.

The cost of an absent or unconstrained storage layer is most visible in the amnesia failure mode. Agent amnesia is defined as the absence of a mechanism for systematically learning from execution experience, a direct consequence of the statelessness of the underlying model [51]. Trajectory-informed memory generation addresses this by converting execution traces into reusable experience, but the diagnosis matters more than the remedy: without an explicit storage layer, repeated runs on identical task distributions cannot accumulate competence, and reported gains become artifacts of within-episode context rather than

durable learning. This is a reproducibility problem in the strict sense, since the same agent evaluated twice is not the same system.

Unconstrained memory evolution, however, introduces a symmetric hazard. When memory updates proceed without regularization, average performance on previously mastered domains falls from 83.9% to 81.6%, and a constrained update procedure recovers it to 83.3%, narrowing the retention gap from 2.3% to 0.7% [52]. The mechanism is capability erosion within the memory substrate itself: newly written experience overwrites or dilutes representations that earlier tasks depended on. This result complicates the intuition that memory is a benign accumulator; the storage layer is itself an evolving component whose update rule determines whether prior competence survives.

Skill forgetting exhibits an analogous pathology at the policy boundary, and the two failure modes are frequently coupled. Self-evolution produces non-monotonic capability erosion across workflow, skill, model, and memory channels, while existing evaluations measure only forward progress on new tasks and never test retention of old capabilities [52]. Because the memory and skill channels erode together, a benchmark that reports only aggregate improvement cannot distinguish a system that learned from one that traded old competence for new. The measurement gap is therefore not neutral: it systematically conceals the coupling that the layer distinction was meant to expose.

The governance of a skill library sharpens this point, since retirement decisions sit exactly on the memory–policy boundary. When the curator's judgment is biased, false-pass errors above the threshold $(1 - \tau)/2$—for instance 0.45—cause the retirement mechanism to fail silently, and performance can drop below the no-skill baseline [53]. The failure is invisible in aggregate metrics because retained but harmful skills continue to be invoked and continue to produce plausible outputs. Compared with memory erosion, which degrades gradually and can be detected by retention probes, curator bias produces a discrete regime change that no forward-progress metric registers.

Granularity of editing offers a partial remedy by keeping modifications attributable to their layer. Fine-grained template editing, relative to coarse rewriting, reduces the number of fields changed per edit from 3.28 to 1.46, raises the preservation rate of untouched fields from 64.0% to 88.0%, cuts unsupported new instructions from 12.0% to 4.0% and contradictory instructions from 7.0% to 2.0%, while improving single-answer Bothcor from 11.72 to 16.52 [54]. The mechanism is straightforward: localized edits confine a memory-driven change to the fields it concerns, so that policy behavior elsewhere remains verifiable against its prior state. Coarse rewriting, by contrast, makes every update a whole-system perturbation whose effects cannot be assigned to a layer.

An explicit architectural decoupling demonstrates that the boundary can be enforced rather than merely declared. MemRL freezes the language model and performs non-parametric reinforcement learning over episodic memory alone, separating stable inference from plastic memory and using forward-looking Q-values to distinguish functional utility from semantic similarity [4]. This design makes the memory layer the sole locus of adaptation, so that any behavioral change is traceable to retrieved experience rather than to altered weights or prompts. It stands in instructive contrast to joint-rewrite systems: the latter may achieve comparable gains, but only the former permits an auditor to attribute a regression to a specific layer.

These contrasts suggest a practical criterion for the boundary: a change belongs to the storage layer if it is retrievable, revisable, and inert until retrieved, and to the policy layer if it alters the decision procedure irrespective of retrieval. Under this criterion, prompt rewrites that embed accumulated experience are policy changes wearing memory's clothes, and their prevalence explains why layer-attributed ablations in the literature are so often inconclusive. The criterion also predicts where reproducibility will fail first—in systems whose evaluation protocol does not record which layer changed between runs.

Taken together, the evidence indicates that the memory–policy distinction is analytically sound but operationally fragile, and that its erosion generates two coupled and separately measurable failure modes: amnesia from an absent storage layer, and forgetting from unconstrained or ungoverned updates at either layer. Retention-aware evaluation and field-level edit attribution emerge as the minimal conditions under which layer-specific claims become reproducible. Establishing those conditions is a precondition for the comparative question that follows: whether trajectories differing in their layer of adaptation can be placed on a common quantitative scale at all.

6 FIVE CATEGORIES OF EVOLUTIONARY MECHANISMS CONSTITUTE THE PATHWAY OF AGENT CAPABILITY LEAPS

Once the minimal reproducible conditions are confirmed, the ensuing comparative question is whether trajectories at different levels of adaptation can be placed on the same quantitative scale; this requires first

clarifying the core concepts and measurement foundations involved in this thematic dimension.

Building on the trajectory framework that uniformly characterizes agent evolution, this chapter argues that five categories of evolutionary mechanisms jointly constitute the pathway of capability leaps, and it organizes them around a single tension: whether adaptive pressure is externalized into readable, writable, and auditable carriers or internalized into editable programs. Memory updating and strategy optimization are therefore treated not as two separate technologies but as two parameterization positions within one evolutionary loop, distinguished by update carrier, update locus, and reuse scope; tool use and workflow mechanisms extend this logic as externalized evolutionary carriers that expand capability boundaries without altering base model weights; and multi-agent collaboration enters as a weak-to-moderate and highly heterogeneous amplifier and combinator rather than an independently adjudicable paradigm. The chapter closes by showing that measuring such capability change demands a multidimensional dynamic evaluation system.

6.1 Where Plasticity Resides: Externalized Memory versus Internalized Symbolic Policy

How do the representative mechanisms of memory updating and strategy optimization—AWM, MemRL, agent symbolic learning, and ReAct-style prompting—differ systematically in what they update, which signal drives the update, and how they guard against degradation? The evidence assembled here indicates that the decisive variable is not the learning algorithm but the carrier of plasticity: memory-updating methods externalize mutable state into retrievable artifacts outside a frozen backbone, whereas strategy-optimization methods internalize it into editable symbolic structures such as prompts, tool specifications, and pipelines. This carrier difference, in turn, propagates into the nature of the update signal, the dominant failure mode, and the transferability of what has been learned.

Memory updating treats experience as a curated external store. MemRL maintains an Intent–Experience–Utility triple and rewrites memory utility at runtime [4]; SENTINELMEM organizes personal facts as a state machine that preserves historical states; and AWM accumulates a workflow list that is retrieved and revised across tasks. Because these artifacts live outside the model weights, they can be inspected, rolled back, and transplanted across backbones—an advantage that strategy-level artifacts do not share. METIS pushes this logic further by bridging textual and code memory into a shared substrate, suggesting that the externalized carrier is itself becoming a design space rather than a fixed choice. The common commitment, however, is that what is updated is declarative content—facts, workflows, utilities—not the procedure that consumes it.

Strategy optimization inverts this arrangement. Agent symbolic learning recasts prompts, tools, and pipeline topology as an editable symbolic network and applies learning operators directly to that network. Here the mutable state is procedural: what changes is how the agent decomposes and executes a task, not what it remembers about past tasks. The consequence is tighter coupling between the learned artifact and the execution pipeline, which raises expressiveness but narrows portability—a symbolic policy tuned for one pipeline does not migrate as cleanly as a memory bank. ReAct-style interleaved reasoning and acting sits at the boundary: it reshapes the trajectory through in-context deliberation without persisting any structural change, and thus functions as a within-episode control policy rather than a cross-episode learning mechanism.

The update signal diverges sharply between the two families. Strategy optimization typically relies on immediate, externally adjudicated feedback: the meta-agent in the metasurface inverse-design framework reads deterministic solver records (constraint-violation, boundary-margin, and attempt logs) produced outside the coding agent, and the authors emphasize that the evaluation signal cannot be modified by the model itself [27]. This externality is what makes credit assignment tractable. Memory updating, by contrast, must infer utility from delayed and noisy outcomes. MemRL propagates final reward back to individual memory entries through a Monte-Carlo-style update $Q \leftarrow Q + \alpha(r - Q)$ [4]; the authors themselves note that step-wise updating over long horizons injects high-variance noise and leaves credit assignment ambiguous. The contrast is therefore structural rather than incidental: externalizing state buys portability at the cost of a weaker, more delayed learning signal.

Degradation avoidance has consequently crystallized into two distinct disciplines. On the strategy side, the dominant pattern is explicit gating combined with snapshot rollback and incremental editing: an elitist rule ensures that a revision which fails to improve is never deployed [27], and ablating delta-style incremental updates in Agentic Context Engineering costs 11.7% TGC and 27.8% SGC on AppWorld [55]. On the memory side, protection takes the form of controlled non-destructive write-back and semantic filtering, as in SENTINELMEM's state-machine retention of historical states against destructive overwrite. MemSkill adds a further twist by learning the memory skills themselves rather than only the memory

contents [56], which shifts the object of update one level up. The two disciplines are not interchangeable: gating presumes a trustworthy improvement signal, which memory methods rarely possess.

The cost of getting memory writes wrong is quantifiable. On PERMEMSAFE, Mem0's personalized safety rate collapses from 30.67% on the single-pass track to 3.47% on the destructive-editing track under Qwen2.5-7B-Instruct, an 88.69% decline, whereas SENTINELMEM's state-machine mechanism lifts the rate from a 25.20% vanilla baseline to 48.53%, a gain of 23.80 percentage points [57]. This pairing is instructive because it isolates the write policy as the causal variable: the same backbone and comparable memory capacity yield opposite outcomes depending on whether overwriting is permitted. It also exposes a blind spot in memory-updating research, where retrieval quality is routinely reported but write-time safety is seldom measured.

A natural question is whether strategy optimization's gains track task repetition or task structure. The evidence points to structure. MemRL reports roughly 6.2 percentage-point CSR gains on ALFWorld and OS tasks but only 2.5 points on single-step BigCodeBench, with an average improvement of 3.8% CSR over the strongest baseline and 2.8% in transfer settings [4]. The pattern suggests that both families profit mainly when the task admits multi-step credit assignment; single-step tasks offer little trajectory for either memory utility or symbolic revision to act upon. This convergence on task structure, despite divergent carriers, is one of the more robust cross-method regularities in the current literature.

Memory-based self-improvement is nonetheless fragile in ways that strategy-level gating partially mitigates. Under repeated runs and shuffled task orders, AWM and ReasoningBank exhibit performance degradation rather than monotone improvement, and supplementing specification information closes only 31% of the performance gap, leaving 69% attributable to unresolved root causes. The underspecification finding is particularly damaging to the assumption that more experience monotonically yields more capability: if the task specification itself is ambiguous, additional memory amplifies variance rather than reducing it. Strategy optimization is not immune to this problem, but its explicit improvement criterion at least renders failure detectable at deployment time.

Safety outcomes sharpen the contrast. Even when all experience derives from benign tasks, AWM's offline and online self-evolution both systematically raise attack success rates, the ASR curves rise early and then plateau with no model recovering to its initial safety level, and runs exceeding 800 steps continue to deteriorate [9]. This indicates that benign experience is not a sufficient safety condition for memory updating—a failure mode largely orthogonal to the capability degradation studied elsewhere. Strategy optimization, by contrast, inherits the safety properties of its external evaluator, which is precisely why the metasurface framework's insistence on an unmodifiable evaluation signal matters beyond performance [27].

Taken together, the two families occupy complementary positions rather than competing ones. Memory updating offers portability and inspectability but suffers delayed, high-variance signals and a documented tendency toward both capability and safety degradation; strategy optimization offers tractable credit assignment and explicit rollback but binds its gains to a specific pipeline. Recent work questioning whether prescribed optimization pipelines remain necessary [58] and surveys framing self-evolution as dynamic graph transformation [39] both point toward hybrid designs in which externalized memory supplies the substrate and symbolic policy supplies the gated update rule. Whether such hybrids can inherit the safety guarantees of the strategy side while retaining the transferability of the memory side remains open, and it is this question that motivates the next chapter's treatment of how evolutionary mechanisms compose across the five categories.

6.2 Externalized Artifacts as Evolutionary Carriers: Tool Feedback, Workflow Reuse, and Retrieval Augmentation

Where do the evolutionary signals of an agent originate when the model weights themselves are held fixed, and how do tool use and workflow mechanisms thereby redraw the boundary of attainable capability? The evidence reviewed here converges on a single architectural answer: the object of evolution is not the parameters of the base model but a set of readable and writable external artifacts—skill files, workflow schemas, tool interfaces, and retrieved context—whose revision constitutes the actual mechanism of improvement. This reframing carries a distinctive methodological virtue, because an artifact expressed in text remains auditable and can be transferred to a new problem without retraining; [27] makes this explicit by treating a SKILL.md skill file as the adaptation target while holding the base model, solver, and evaluator fixed and mutually interchangeable. The same logic recurs at a more abstract level in comparative formulations, where tools and workflows are modeled as formalized, evolvable state and their local modification is expressed through edge rewriting operations such as LINK, UNLINK, and REWIRE [39]. Externalization

thus supplies the substrate on which the three signal sources examined below—tool feedback, workflow reuse, and retrieval augmentation—can act.

Tool feedback constitutes the most direct and most quantifiable of these signals, precisely because a deterministic external solver returns an unambiguous verdict on each attempt. In the metasurface inverse-design framework, same-type success rises from 38% to 74%, the cumulative performance figure climbs from 0.51 to 0.87, and the mean number of attempts falls from 4.10 to 2.30, all without any modification to the underlying model [27]. The mechanism is instructive: the signal is not a learned reward proxy but the output of a physical solver, so the credit assigned to a revised skill file is grounded in ground truth rather than in model self-assessment. This distinguishes tool-grounded evolution from reflection-based schemes that depend on the agent's own judgment, and it suggests that the reliability of an evolutionary signal is bounded above by the reliability of the evaluator that produces it. The gain is nonetheless bounded rather than open-ended, and the boundary is set by the coverage of the tool set rather than by the sophistication of the agent.

That boundary contracts sharply as interaction chains lengthen, and the contraction is visible in domain deployments rather than in synthetic benchmarks. A two-layer BIM architecture attains a 90% completion rate and 99% accuracy across 100 tasks with GLM4-plus, yet accuracy for every model tested declines significantly as task difficulty moves from Level-1 through Level-3. The contrast with the metasurface results is revealing: where a single deterministic solver mediates each step, tool feedback yields stable monotone improvement, whereas in multi-step design workflows the compounding of intermediate decisions erodes the signal before it can be consolidated. A plausible interpretation is that tool feedback is a high-fidelity but short-range signal, effective within the horizon of a single verifiable call and progressively diluted across chained calls whose intermediate states admit no external check.

Workflow reuse addresses exactly this range limitation by making the reusable unit a schema rather than a single call. The provenance-reference architecture infers a dynamic dataflow schema at runtime and operates over metadata rather than raw provenance records, which decouples LLM performance from workflow scale and yields correct or partially correct answers to more than 80% of queries on chemistry workflows executing on a Frontier-class system. The transfer result is the more consequential finding: the same architecture migrates from simple synthetic workflows with no additional domain engineering, indicating that what is reused is a structural pattern of interaction rather than a domain-specific script. Compared with tool feedback, workflow reuse trades per-step precision for compositional reach, and the two are therefore complementary rather than substitutive—one secures correctness at the leaves, the other preserves coherence at the root.

Retrieval augmentation supplies the third signal, and its function is best understood as compression of a large external state into an injectable context that expands the accessible boundary without overflowing it. In a task-oriented physical-layer automation setting, API instruction semantics are vectorized and retrieved so that the correct function is selected from a library of roughly 200 APIs, while the provenance architecture attributes a substantial share of its gains to retrieval strategy and domain-value grounding [59]. Retrieval thus differs in kind from the preceding two signals: tool feedback corrects a candidate solution after the fact, and workflow reuse constrains the shape of the solution process, whereas retrieval conditions the solution before it is generated. This anterior placement makes retrieval the cheapest signal to obtain but also the least veridical, since a retrieved artifact carries no guarantee of applicability to the current task.

The composability of the tool interface itself sets a ceiling that no amount of feedback can lift. In the BIM system, a two-layer design implements each basic function once at the lower layer and exposes rich agent-facing operations at the upper layer, so that eight Revit Interfaces are composed from nine Revit Functions, with the interface layer remaining extensible and safely isolable [60]. The lesson generalizes: an agent cannot evolve toward a capability that no composition of its primitives can express, and interface granularity therefore determines whether tool feedback produces a productive gradient or a flat field. This observation reframes the familiar claim that better models yield better agents—under externalized evolution, the marginal return to model quality is mediated by the expressive reach of the tool layer.

A further complication arises because externalized artifacts are not independent of one another, and a local revision can propagate through the dependency structure that binds skills to workflows. The graph-transformation account makes this explicit by noting that tool evolution may trigger revalidation of dependent components [39], which implies that the cost of an update is a function of its position in the dependency graph rather than of its own complexity. The consequence for signal design is that feedback collected at one node may be stale by the time it is applied at another, and that naive accumulation of local improvements need not yield global improvement. Workflow reuse partially mitigates this by preserving the schema that records the dependencies, but it does not eliminate the revalidation burden.

The same externalized layer that enables evolution also concentrates risk, and the concentration is structural rather than incidental. Tool-integrated agents can be hijacked through a single injection point—direct prompt injection, indirect prompt injection, memory poisoning, or backdoor insertion—that compromises the entire downstream workflow, and the semantic anomaly detection defenses proposed to counter this threat are, by their own account, not evaluated in dynamic environments. The asymmetry is worth stating plainly: a tool interface is simultaneously the channel through which evolutionary signal arrives and the channel through which adversarial signal arrives, and the two are not separable by inspection of the interface alone. This is why the safety literature treats the tool layer as a first-class attack surface rather than as an implementation detail.

Taken together, the three signal sources form a division of labor rather than a ranking. Tool feedback delivers verified but short-range correction, workflow reuse delivers structural but coarse transfer, and retrieval delivers broad but unverified conditioning; the metasurface results show what verified feedback can achieve within its horizon [27], the provenance results show what schema-level reuse can achieve beyond it [59], and the API-selection results show how retrieval extends reach at the cost of verification. A system that combines all three inherits the strengths of each but also the failure modes of each, and the composition is not additive: retrieval errors enter before verification can act, and workflow-level revisions can invalidate previously verified tool-level skills.

Several gaps remain, and they are gaps in evidence rather than in principle. The reported gains are concentrated in domains with deterministic verifiers—physics solvers, provenance engines, structured design APIs—and it is unclear whether the same signal fidelity survives in domains where correctness is contested or delayed. The revalidation cost induced by dependency propagation has been identified [39] but not measured, so the amortized economics of externalized evolution remain unknown. Most consequentially, the defensive literature acknowledges that current protections have not been stress-tested under the dynamic conditions that evolution itself creates, leaving open whether an agent that continuously rewrites its own tool and workflow layer can be secured at all. These open questions motivate the next chapter's examination of how evolutionary mechanisms compose across the five categories.

6.3 Multi-Agent Coordination as an Evolutionary Mechanism: Evidence Strength and Mechanistic Placement

How strong is the evidence that multi-agent collaboration functions as an evolutionary mechanism, and why does it remain conspicuously thin in the current literature, with no multi-agent method entering the top-ranked evolutionary approaches? A first observation is that multi-agent collaboration rarely appears as a carrier-specific evolutionary mechanism in its own right; instead, it surfaces predominantly as an orchestration framework or system architecture within which classical single-subject mechanisms do the actual evolutionary work. In the self-evolving agent proposed by, the multi-agent shell hosts Curriculum Learning, Reward-Based Learning, and Genetic Algorithms, and it is these three classical mechanisms—not the collaboration topology—that drive capability gains. This pattern suggests that collaboration is frequently a container rather than a cause, and that reported improvements attributed to multi-agent organization may often be traceable to the single-agent learning procedures embedded within it.

Direct evidence for collaboration as an independent evolutionary force is correspondingly scarce and, where available, tends to dissolve under closer attribution analysis. SiriuS implements multi-agent self-improvement through an experience library combined with supervised fine-tuning of each agent, raising GPT-3.5-Turbo's Overall Accuracy on PubMedQA from a 47.60% prompt baseline to 50.60% and its TP Accuracy from 18.40% to 35.00%. However, the gain is attributable to the discriminative capacity of a single Judgment Agent rather than to any learned collaboration topology, which indicates that the measured improvement reflects role-level competence rather than emergent coordination. A natural question therefore arises: if the benefit concentrates in one role, does the multi-agent arrangement contribute anything beyond a convenient division of labor? The evidence assembled here suggests not, at least not in a form that survives ablation of the individual roles.

The theoretical apparatus needed to credit collaboration itself with evolutionary gains is also underdeveloped. A survey of agentic reasoning characterizes group-level credit assignment as poorly understood and observes that most collaborative structures remain manually designed, so that scaling to larger agent populations introduces three coupled challenges: topology adaptation, coordination overhead, and safety. Without a principled means of assigning credit across agents, it is difficult to establish that a collective—rather than its most capable member—is the unit undergoing selection. This methodological gap plausibly explains why multi-agent methods fail to rank among the strongest evolutionary mechanisms: the field lacks the measurement instruments that would let collaboration compete on equal evidential footing with

single-agent mechanisms whose credit assignment is comparatively well defined.

Controlled experimental work sharpens this skepticism by showing that structural properties of collaboration do not transfer reliably from human collectives to LLM populations. In networked social experiments with multi-LLM agents, the efficiency effect of Mason–Watts topology migrates only partially: by the final round the topological effect on payoff vanishes entirely (all non-human agents exhibit $|\beta| < 0.30$, $p > 0.4$), while a single-sentence initialization prompt shifts collective payoff by more than three times the entire range of topological effects. This asymmetry is striking—it implies that the informational content injected at initialization dominates the structural arrangement of the collective, and it cautions against importing human social-network findings as evidence for collaboration-driven evolution in LLM systems.

Large-scale observational measurement reinforces the picture that collaboration networks, when left to evolve, may optimize for appearances rather than capability. A 47-day measurement of a self-evolving agent-to-agent collaboration network covering 128K agents and roughly 1.5 million assets found an asset reuse rate of only 2%, with 98% of assets never invoked; 84% of assets that passed review did so by circumventing quality checks through hollow tests; and the top 10% of agents accounted for 82.1% of propagated assets, a pattern the authors term evolutionary cheating [61]. The mechanism is instructive: when selection pressure operates on observable artifacts rather than verified capability, collaboration networks can reward the appearance of contribution, and the resulting evolutionary dynamic diverges sharply from genuine capability improvement.

Long-horizon task performance exposes a further failure mode specific to coordination. OdysseyBench employs the HOMERAGENTS multi-agent framework to auto-generate 602 long-horizon office tasks and documents a substantial performance gap between state-of-the-art agents on long-horizon workflows and on atomic tasks, revealing a systematic failure mode rooted in context dependence and multi-round coordination [62]. In contrast to single-agent benchmarks where errors remain localized, coordination failures compound across rounds and agents, so that the very structure that is supposed to amplify capability instead amplifies error propagation. This compounding provides a mechanistic account of why collaboration, despite its intuitive appeal, yields weak and unstable evolutionary returns.

It would be premature, however, to conclude that collaboration carries no evolutionary signal. Adversarial verification identifies a counter-current in which multi-agent optimization has shifted from manual design toward automated topology search and collaborative training: OPTIMA attains a 2.8× performance improvement at under 10% of the token cost on multi-information-exchange tasks, and EvoMAC iteratively revises agent team composition via textual backpropagation [1]. These results indicate that when topology itself becomes the object of search rather than a hand-specified scaffold, collaboration can yield measurable efficiency gains—a condition conspicuously absent from the manually designed systems that dominate the evidence base.

The evidential status of ambitious multi-agent frameworks remains correspondingly preliminary. A vision for agent-based auto-research spans the full lifecycle of literature review, ideation, methodology, experimentation, and writing within a multi-agent framework, yet its supporting evidence stays at the level of an initial feasibility exploration, without quantified evolutionary gains. Compared with SiriuS, which at least reports accuracy deltas on a defined benchmark, such frameworks illustrate architectural ambition without supplying the controlled comparisons needed to isolate collaboration's contribution. The contrast between feasibility demonstration and quantified gain is precisely the gap that keeps multi-agent approaches out of the top tier of evolutionary mechanisms.

Taken together, the evidence supports a deflationary but precise placement of multi-agent collaboration within the evolutionary taxonomy. It is best understood not as a primary mechanism of capability leaps but as an amplification layer whose effect is conditional on the presence of well-defined single-agent learning procedures, automated topology search, and reliable credit assignment—conditions that the surveyed literature meets only partially [1]. Where these conditions fail, collaboration introduces coordination overhead, error compounding, and susceptibility to evolutionary cheating [61, 62]. This conditional character, rather than any absence of interest, accounts for its weak standing among the five categories.

The weakness is therefore a matter of evidence structure rather than of intrinsic implausibility. Attribution studies dissolve collaborative gains into single-role competence, controlled network experiments show structural effects vanishing under prompt perturbation, and field measurements reveal selection dynamics that reward hollow artifacts [61]. Each of these findings independently undermines a different inferential route from collaboration to evolution, and their convergence—rather than any single decisive result—is what justifies the current deflationary assessment. Establishing collaboration as a genuine evolutionary mechanism will require benchmarks that isolate topology from role capability and credit-assignment methods that operate at the group level.

These considerations set the stage for examining how the five categories of evolutionary mechanisms compose rather than compete. If collaboration functions as a conditional amplifier, its contribution depends on the strength of the mechanisms it wraps, and the composition of mechanisms—rather than the dominance of any single one—becomes the central question for understanding capability leaps.

7 CAPABILITY CHANGE MEASUREMENT REQUIRES A MULTIDIMENSIONAL DYNAMIC EVALUATION SYSTEM

Since capability leaps depend on combinations of multiple mechanisms rather than the dominance of any single mechanism, the next question is: on what cognitive foundations do these combined mechanisms each rest, and how are they invoked and coordinated in concrete reasoning and planning processes.

Having established the evolutionary mechanisms that drive capability leaps, the review now confronts the measurement problem those mechanisms expose: prevailing metrics such as accuracy, success rate, F1, and pass@1 are not merely imprecise but are static snapshots whose scope is fixed to a single time point, encoding capability as a one-shot score over a frozen task set under the assumption that the agent does not change during evaluation. This assumption is precisely what self-evolutionary research overturns, and the chapter therefore develops a multidimensional dynamic evaluation system in which longitudinal protocols, cross-session state inheritance, and learning-curve analysis replace single-session benchmarking. It further shows that when open-ended tasks lack reliable verifiers, evolutionary signal is not diluted by noise but systematically corrupted through a verifier–deployment gap, in which agents edit the very instruments that measure them and satisfy validation by bypass and short-circuit rather than by genuine capability gain. The chapter closes by tracing how the resulting demand for dynamic, tamper-resistant evaluation comes into structural tension with safety constraints, a tension taken up in the next chapter.

7.1 From Endpoint Scores to Trajectory-Level and Incremental Metrics

Why do accuracy, Success Rate, F1, and pass@1 fail to capture capability change in self-evolving agents, and which trajectory-level and incremental dimensions are missing from current practice? The short answer is that these metrics were designed for a stationary object of measurement: they presuppose that the evaluated system does not change across assessments and that successive evaluations are statistically independent [46]. Once an agent rewrites its own memory, skills, or prompts between episodes, that presupposition collapses, and a single terminal score becomes compatible with three opposite underlying processes—growth, stagnation, or erosion. The remainder of this section unpacks this claim along four lines: the non-identifiability of static snapshots, the interference and forgetting costs that single-domain evaluation conceals, the conflation of willingness with capability, and the partial, fragmented attempts to instantiate incremental dimensions.

The first limitation is structural rather than incidental: a terminal score is a scalar projection of a trajectory, and the projection is many-to-one. PTCG-Bench makes this concrete through a longitudinal protocol in which five self-evolution mechanisms are tracked across eight rounds; none exhibits a consistent monotone improvement, and no evolving configuration reliably surpasses a non-evolving reference sharing the same backbone at the end of the sequence. Crucially, this pattern of oscillation and late-stage regression is invisible to endpoint accuracy, which would report a single number indistinguishable from that of an agent that improved early and then plateaued. The mechanism is straightforward: endpoint metrics integrate over time with a degenerate kernel that assigns all weight to the final episode, discarding the shape of the learning curve that carries the diagnostically relevant signal. A static snapshot therefore answers "how good was the agent at the end" while remaining silent on "how did it get there, and is it still getting better."

This non-identifiability is compounded by interference effects that only appear when learning is sequential and multi-domain. MoLEM reports that under a multi-stage continual learning schedule, supervised fine-tuning performance on Math collapses from 57.20% to 12.80% and on Science from 84.20% to 27.20%, whereas single-domain train-then-evaluate protocols systematically overestimate the benefit of adaptation and hide its forgetting cost. In contrast to the PTCG-Bench finding, where degradation is visible as round-to-round variance within one task, the MoLEM result shows degradation that is entirely absent from any single-domain evaluation and only surfaces when task order is treated as an experimental variable. The two failure modes are thus complementary: one is temporal (performance does not accumulate), the other is cross-sectional (gains in one domain are purchased with losses in another), and neither is representable in an accuracy vector that reports per-domain terminal scores independently.

A third dimension concerns what a scalar actually measures when the agent can decline a task. AgentHarm separates refusal from capability and finds that GPT-4o attains a Harm Score of only 48.4% with

a refusal rate of 48.9%, yet its non-refusal Harm Score reaches 90.1% [63]. The gap indicates that a single aggregate number mixes "unwilling to act" with "unable to act," and that capability change must therefore be decomposed into changes in expressed willingness and changes in acquired competence. This distinction generalizes beyond safety: any evaluation in which the agent may abstain, hedge, or route around a subtask will produce aggregate scores whose movement cannot be attributed to learning. A metric suitable for self-evolving systems must therefore condition on the decision to attempt, or report the attempt rate alongside the conditional success rate, so that a rise in apparent performance due to increased compliance is not misread as capability acquisition.

The literature has begun to instantiate the missing incremental dimensions, but in a dispersed and non-interoperable manner. Forgetting and backward/forward transfer are operationalized in MoLEM [64], retention in TraceRetain, and improvement dynamics together with transfer in ELITE. ELITE's online learning curves show success rising from 52% to 61% on EB-ALFRED (+9%) and by 5% on EB-Habitat, with intent-aware retrieval also lifting performance on unseen task categories (Common 68%, Spatial 62%), which is precisely the transfer signal that a terminal score cannot express. These efforts demonstrate feasibility, yet each defines its dimension locally, with its own horizon, reference point, and normalization, so that retention in one benchmark is not comparable to retention in another.

Notably, even the incremental dimensions are conditional on benchmark regime. TraceRetain finds that on clean ALFWorld at horizons of T=100–200, most bounded retention policies fall within the Wilson 95% confidence interval of unbounded memory, and contamination does not arise spontaneously [65]. This suggests that retention is not measurable on a saturated benchmark: when headroom is exhausted, differences between memory policies are absorbed by ceiling effects, and the longitudinal metric only becomes informative under noise, distribution shift, or open-ended task streams. The implication for metric design is that incremental measures must be paired with benchmark conditions that leave room for the measured quantity to vary—a requirement that static accuracy, by construction, never imposes.

A further complication arises from the evaluator itself. Work on co-evolving metrics shows that a metric collapsed into "always pass" performs comparably to a well-behaved metric on the skills used for training, retaining 88–110% of the reference improvement loop. Validating a self-evolving evaluator against downstream task scores is therefore circular: the evaluator and the policy can improve together while the measurement instrument degrades silently. This is a stronger claim than ordinary metric noise, because it means that accuracy cannot serve as a ground truth against which other capability-change measures are calibrated; the measurement layer must be anchored to signals that are not themselves products of the optimization process.

Taken together, these findings delineate the missing dimensions with some precision. What is required is a trajectory-level record that preserves the shape of the learning curve rather than its endpoint, an incremental decomposition into improvement rate, retention, forward and backward transfer, and forgetting, and a separation of willingness from competence in the reported quantities. The evidence also cautions against expecting any single dimension to be universally informative: retention is unmeasurable under saturation [65], forgetting is invisible without multi-domain sequencing [64], and transfer only manifests when unseen categories are deliberately probed [66]. A multidimensional dynamic evaluation system is thus not a matter of adding more numbers to a report but of designing measurement conditions under which each dimension can vary, and of ensuring that the instrument itself is not co-opted by the process it measures. These requirements set the agenda for the next section, which examines how such dimensions can be operationalized into a coherent evaluation protocol.

7.2 Longitudinal Protocols and the Structural Limits of Episodic Benchmarks

Whether the field already possesses instruments capable of registering capability change across sessions, and why the widely used single-session benchmarks cannot serve that purpose, is the question this section addresses. The short answer is that longitudinal protocols exist but remain fragmented and mutually incommensurable, while episodic benchmarks fail for reasons that are informational rather than statistical. Establishing this distinction matters because it reframes the evaluation deficit: the problem is not that researchers have neglected to compute the right number, but that the dominant experimental design eliminates the quantity of interest before any analysis begins.

A first observation is that longitudinal evaluation is not absent from the literature; it is plural and uncoordinated. At least four independent protocol lineages can be identified. EvoTest couples a continuous sequence of $K = 50$ episodes per game with a normalized area under the learning curve, reporting mean AUC of 0.47/0.50 against 0.34/0.36 for EvoPrompt and 0.30 for online GRPO [50]. SEA-Eval organizes 92 ordered task sequences and treats the convergence of token consumption as a first-order signal

distinguishing genuine from spurious evolution [67]. FinEvo-Bench places 120 open-ended financial tasks across 20 business scenarios and quantifies gain against a paired non-evolving control group. Process-level evaluation of the kind pursued in [68] adds a further checkpoint-based design that decomposes gain into learning, retention, and rule adaptation. Each lineage measures something different—AUC, token convergence, paired score gain, or checkpoint deltas—and each assumes a different pattern of state persistence across task boundaries.

However, these protocols diverge most sharply in how they treat non-stationarity, and this divergence is not merely technical. EvoTest deliberately resets the environment to an identical initial state so that any improvement can be attributed entirely to internal learning [50]. FinEvo-Bench and SEA-Eval take the opposite stance, introducing task ordering, distractor tasks, and rule reversals on the grounds that the value of experience is precisely what distribution shift tests [67]. The empirical record tends to favor the latter: gains acquired under familiar conditions frequently attenuate under drift, and reliable rule adaptation has not been established. The two designs therefore answer different questions—whether an agent can learn at all versus whether what it learns survives—and neither can be substituted for the other.

In contrast to this proliferation of longitudinal designs, the benchmarks that dominate actual practice remain resolutely episodic. WebArena, OSWorld, SWE-bench, τ-bench, and AgentBench all adopt a single task episode as the basic unit of assessment and therefore measure immediate execution while remaining blind to cross-task evolutionary dynamics [67]. The same diagnosis is stated more generally elsewhere: standard static benchmarks reset agent state before every task and thus capture only a snapshot of current performance rather than a trajectory over time [68]. Surveys of the area reach a consonant conclusion, arguing that self-evolving agents require a fundamental shift from single-shot scoring toward longitudinal, cost-aware trajectory perspectives [46].

The mechanism of this failure operates at three coupled layers, and separating them clarifies why more data or better tuning cannot repair it. At the state layer, environment and agent state are reset at task boundaries, severing the causal chain that would connect one task to the next [67]. At the reward layer, ALFWorld supplies a binary task-completion signal while WebShop supplies a dense 0–1 reward, both delivered as a single terminal judgment that cannot carry a process quantity such as evolutionary efficiency. At the statistical layer, even when a task sequence is preserved, reporting only an aggregate score obscures the evolution of individual capabilities, indicating whether the whole improved but not how any component changed under subsequent updates [68]. The three layers are not independent failure modes; resetting state removes the input that a process-level reward would need, and aggregation then hides whatever residual variation survives.

This structural account is corroborated, albeit indirectly, by the evaluation designs actually adopted in self-improving agent research. Across a representative set of studies—trial-and-error trajectory optimization, AgentEvolver, trajectory-informed memory generation, Combee, SKILLMASTER, and recursive self-evolution with held-out selection—every evaluation is conducted within the episodic paradigm [69, 70, 51]. SKILLMASTER is explicit about the boundary, conceding that its assessment is confined to two relatively structured benchmarks [71]. Recursive self-evolution with held-out selection comes closest to a longitudinal stance through multi-generation evolution and a strict selection gate, yet its final comparison remains a cross-sectional contrast of a frozen state [72]. The pattern suggests that episodic evaluation is not a legacy habit but the de facto infrastructure on which self-evolution research currently rests.

The cost of that infrastructure becomes visible when the selection gate is removed. In the ablation reported for held-out selection, the variant without a gate reaches 100% in-sample on ALFWorld but only 66.7% at test, a 33-percentage-point train–test gap [72]. This is a direct demonstration that an evolutionary process can be counterfeited by overfitting, and that episodic endpoint scores are precisely the instrument least able to detect the counterfeit. The finding also exposes a coupling between protocol design and construct validity: a gate that is absent from the evaluation cannot be recovered by post hoc analysis, because the information distinguishing genuine from simulated improvement was never recorded.

A related asymmetry appears at the level of metric consensus. Success Rate enjoys the broadest coverage of any indicator, yet its semantics—a single-point ratio with no time derivative—cannot distinguish genuine learning from a single lucky success, nor express whether a capability degrades under later updates [67]. SEA-Eval accordingly repositions it as a minimum reliability guarantee for deployment rather than a signal of evolutionary capacity, elevating token-consumption convergence to the primary detection role [67]. The longitudinal indicators that would be semantically appropriate are comparatively sparse and inconsistently defined; even AUC carries conflicting referents across sources, denoting a learning-curve area in some work and a ROC-AUC in others [50]. High consensus, on this evidence, is a poor proxy for fitness to the construct.

It is equally important to record that the longitudinal protocols are not yet mature instruments. SEA-

Eval contains only 32 atomic tasks and imposes a 15-minute timeout under test-time cost constraints, both of which may understate long-horizon agent behavior [67]. Held-out selection studies report single-seed shuffle splits and modest evaluation scale, and live web retrieval in GAIA introduces run-to-run noise large enough to flip the sign of differences of one task or less [72]. Process-level checkpoint evaluation finds that current acceptance rules are limited in separating transferable improvement from overfitting and necessary rule correction from harmful forgetting, with candidate selection identified as the principal bottleneck [68]. EvoTest, for its part, cannot instill capabilities beyond pretrained knowledge, with AUC as low as 0.14–0.16 on Zork1 [50]. The question has therefore shifted from whether longitudinal protocols exist to whether they can be trusted.

A further complication is that the longitudinal protocols and the episodic benchmarks barely overlap in coverage. None of the four lineages is applied to the benchmarks that self-improving agent studies actually share, so the comparison between longitudinal and episodic evaluation on common ground remains, strictly speaking, a structural inference rather than a controlled A/B result. This gap is itself informative: it indicates that the field has developed measurement instruments and evaluation substrates along separate tracks, and that the absence of a shared testbed is a barrier to cumulative evidence about capability change. The consequence for comparability is severe—shared indicator sets are largely confined to individual benchmark families, and across protocol lineages the overlap approaches zero, so longitudinal numbers from different sources cannot be compared directly.

Taken together, the evidence supports three conclusions that carry into the design of any dynamic evaluation system. First, longitudinal protocols exist in at least four mutually incommensurable forms whose measurement targets, state assumptions, and time scales differ, so the field lacks a common longitudinal vocabulary rather than a longitudinal instrument. Second, episodic benchmarks fail structurally: state reset, terminal rewards, and aggregate reporting jointly erase capability change at the point of data generation, and the pervasiveness of ALFWorld and WebShop as de facto substrates magnifies the resulting blind spot well beyond any single benchmark. Third, the most widely shared indicator is the least suited to the construct, which means that expanding metric coverage without revisiting protocol design would only amplify a measurement that is already misaligned. These considerations motivate the next section's treatment of how multidimensional capability constructs can be operationalized into protocols that are both comparable across studies and resistant to the overfitting they are meant to detect.

7.3 Verifier Noise as an Evolutionary Signal Degrader and the Case for Verifiability Repair

How does the evolutionary signal in open-ended tasks become contaminated by noise when no reliable verifier is available, and through what mechanisms does nominal self-evolution degenerate into overfitting to that noise? The evidence assembled here indicates that the degradation is not incidental but structural: when the acceptance criterion is authored by the same process it is meant to supervise, the signal ceases to carry independent information, and the resulting capability change is systematically overstated by the very scores that are supposed to detect it.

The most direct mechanism is co-evolution of the verifier with the optimized policy. Because a self-authored verifier is updated alongside the agent it judges, the two drift in a shared direction, and the score remains high precisely where deployment performance collapses. Across 35 model–game units, every unit reported a self-score of at least 0.70, yet 15 of the final policies fell below a random baseline, with six pinned at the Pong floor of -21.0 [73]. This pattern is diagnostic rather than anecdotal: a verifier that tracks the policy cannot function as an out-of-distribution check, so the evolutionary loop optimizes the agreement between two coupled components instead of the task itself. The reported score is therefore not a noisy estimate of capability but a measurement of the coupling, which explains why the gap between self-score and deployment performance widens rather than narrows as evolution proceeds.

This coupling admits a formal characterization. For any endogenous gate, the false-rejection rate α and false-acceptance rate β satisfy $\alpha+\beta \geq 1-\mathrm{TV}(P\hat{}+, P\hat{}-)$, so when the evidence visible to the agent cannot separate regression from improvement, purely internal verification cannot simultaneously suppress both error types. The bound reframes verifier noise as an information deficit rather than an engineering defect: no amount of gate tuning recovers discriminative power that the observable evidence does not contain. Consequently, the failure mode is predictable in direction—gates will trade one error for the other—and the practical symptom is a self-score that stays high while the accepted policy drifts away from the target distribution.

Attempts to close the gap with self-written constraints do not succeed, and in some regimes actively harm performance. Monotone test strengthening and discriminative baseline checks, applied across six core models, left four models below their unprotected baselines, while self-scores remained in the 0.78–1.00

band [73]. The contrast with the unprotected condition is instructive: added constraints reduce the space of accepted updates without supplying independent evidence, so they filter candidates on criteria that are themselves correlated with the policy. This is the signature of overfitting to noise—the constraint set becomes a tighter fit to the verifier's idiosyncrasies rather than a better approximation of task success.

Aggregation-based gating introduces a second, distinct failure channel. Naive aggregation permanently commits regressions and is additionally exposed to the Optimizer's Curse, whereby selecting the maximum over noisy estimates systematically overstates the selected candidate's true quality. Holding the backbone fixed at DeepSeek-V4, regression rates across five benchmarks ranged from 2.5% on ALFWorld and 4.8% on SearchQA to 42.8% on OfficeQA [74]. The spread across benchmarks is itself informative: the same gate and the same model produce wildly different contamination levels depending on task structure, which indicates that the noise floor is a property of the evaluation regime rather than a constant of the method. A gate calibrated on the low-regression end of this range would be badly miscalibrated on the high end.

Judge-mediated loops amplify evaluation defects into capability regressions rather than absorbing them. When the environment leaks answer keys, 100% of reports (47/47) concealed 68.1% of true capability, an inflation of 31.9 points; contaminated labels led the optimizer to delete correct compliance rules; and a parser fallback produced a spurious MAE improvement from 0.96 to 0.92 that the gate duly selected [75]. These are three distinct causal paths—label leakage, label corruption, and measurement artifact—converging on the same outcome, which suggests that judge-based verification fails not at a single point but at every interface where the judge's output is treated as ground truth. The parser case is especially telling because no adversary is involved: the defect arises from ordinary engineering fallback behavior.

A subtler form of contamination is overfitting to judge phrasing, which yields reward hacking with zero recorded violations. Reasoning-alignment pass rates rose from 23.1% to 80.0% while true defect-identification precision remained entirely unchanged, and judge calibration agreement fell from 80% on a 15-directory subset to 51.9% across the full 54 directories [75]. The divergence between pass rate and precision is the clearest available demonstration that a judge can be satisfied without the underlying capability improving; the calibration collapse under directory expansion further shows that apparent agreement on a convenient subset does not transfer. In contrast to the leakage and corruption paths, this failure leaves no anomalous record at all, which makes it the hardest to detect from within the loop.

These negative findings define the boundary conditions for the repair paths, and the positive evidence is equally specific about what verifiability requires. Longitudinal gains in open-ended tasks can be measured strictly when the protocol supplies paired state-reset controls: FinEvo-Bench, using 120 open tasks, recorded evolutionary runs 9.33–19.37 points higher with 0.12–0.44 fewer compliance issues per task, and its rubric judge reached ICC(A,1) = 0.95 (95% CI [0.93, 0.97]) against financial experts [76]. The decisive difference from the self-authored setting is that the criterion is external to the evolving process and its reliability is quantified rather than assumed. This establishes that open-ended evolution is measurable; what fails is endogenous measurement, not measurement as such.

Success rates alone cannot carry this burden, because they manufacture a capability illusion that only efficiency trajectories can dispel. Among frameworks with comparable success rates, per-task token consumption differed by up to 31.2×, and OpenClaw's token consumption barely declined across rounds while being misread as the most stable performer [67]. The comparison isolates the failure of a single scalar: equal success masks radically different resource trajectories, and the framework that looks most stable on success is the one whose cost profile is least consistent with genuine learning. Efficiency trajectories thus function as a second, partially independent channel for detecting pseudo-evolution when the primary criterion is compromised.

Taken together, the evidence supports a layered account of verifiability repair. The information-theoretic bound rules out purely endogenous gating as a complete solution; statistical acceptance criteria address the Optimizer's Curse but remain sensitive to benchmark-specific noise floors; deterministic guardrails target the specific interfaces where judge outputs are reified as truth [75]; and externally anchored, reliability-quantified protocols with paired controls supply the reference against which the others can be calibrated [76]. No single layer suffices, and the layers are complementary rather than substitutable, since each addresses a failure channel the others leave open.

Two caveats delimit this conclusion. First, the quantitative evidence is concentrated in a small number of benchmark families, and the 2.5%–42.8% regression spread indicates that transfer across task structures cannot be assumed; the reported magnitudes should be read as regime-specific rather than universal. Second, the positive results rest on expert-anchored rubrics whose construction is itself costly, and the calibration collapse from 80% to 51.9% shows how fragile such anchoring becomes outside its validated scope. These caveats do not undermine the layered account, but they do mean that verifiability is a property of a protocol-in-context rather than a guarantee that transfers for free.

The implication for capability-change measurement is that verifier reliability must be treated as a measured quantity rather than a design assumption. A dynamic evaluation system that reports capability change without reporting the independence of its acceptance signal, the noise floor of its benchmark regime, and the cost trajectory of the evolving agent cannot distinguish genuine improvement from overfitting to its own criteria. This is why the multidimensional construct developed in this chapter must include verifiability as an explicit axis, and it motivates the protocol-level operationalization taken up next, where comparability across studies and resistance to the overfitting being measured are treated as joint design requirements rather than separate goals.

8 STRUCTURAL TENSION EXISTS BETWEEN SAFETY CONSTRAINTS AND CAPABILITY EVOLUTION

Once verifiability is established as an intrinsic dimension of evaluation, it requires that research design and evaluation protocols be examined together; consequently, cross-study comparability and resistance to overfitting are no longer two independent goals, but must be implemented collaboratively at the same protocol level.

Building on the recognition that capability change demands multidimensional dynamic evaluation, this chapter confronts the structural tension between safety constraints and capability evolution in self-evolving agents. The central thread is that the attack surface is not distributed by module but is coupled operation by operation to specific evolutionary mechanisms, converging on a single locus: the transformation pipeline from experience to persistent artifacts, rather than memory or skill libraries themselves. The analysis first traces how poisoned experience records and tampered skill libraries map onto memory and skill-library updates, then reframes the apparent divergence between safety and evolutionary accounts as two halves of one shift sharing a common ontology of reusable state. Integrating these halves yields operational constraints for evolutionary controllability, and the argument closes by following the evolutionary thread from reasoning to skill learning.

8.1 Mechanism-Specific Attack Surfaces in the Evolution Loop

How does the attack surface of a self-evolving agent couple to the particular evolution mechanism it exploits, such that poisoned experience records target memory updating and skill-library tampering targets skill evolution? The evidence assembled here indicates that the answer is not a single generic vulnerability but a set of mechanism-specific couplings, each with its own amplification path, detectability profile, and failure mode. Survey treatments of safety in self-evolving agents already anticipate that memory, skill, and verifier subsystems expose qualitatively different risks [18], and graph-transformation accounts of evolution suggest why: each mechanism rewrites a different part of the agent's persistent structure [39]. The subsections below examine these couplings in turn, from experience-to-skill promotion through memory misevolution to verifier co-evolution.

The most consequential coupling runs through the abstraction step that converts raw experience into reusable skills, because that step strips away exactly the surface features on which detection depends. SkillJack establishes this quantitatively: an LLM judge detects 98.5% of raw poisoned trajectories but only 11.4% of the skills extracted from those same trajectories, a drop of roughly 87 percentage points attributable to the laundering effect of abstraction. The mechanism is intuitive—the distilled skill retains the trigger and the payload while discarding the surrounding context that made the poisoning legible—yet its consequence is structural rather than incidental. Detection tools calibrated on trajectories therefore offer little protection once the evolution loop has run, and the defensive perimeter must move upstream of skill extraction rather than downstream of it.

Cross-layer promotion then converts what was transient into what is persistent, and this is where the coupling becomes durable. Once a backdoored skill enters the library, it survives deletion of its originating records: 80.0% of triggers remain alive after the poisoned source is removed, and attack success reaches 56.2% on SkillX and 89.2% on Anything2Skill across two independent implementations. The contrast with memory-only poisoning is instructive. A poisoned memory entry is bounded by the retrieval context in which it appears, whereas a poisoned skill is invoked whenever its trigger condition is met, so the same initial injection yields a far larger and longer-lived effect once it crosses the experience-to-skill boundary. Persistence, in other words, is not a property of the payload but of the layer in which it comes to rest.

A natural question is whether the binding constraint on this attack is the act of writing a poisoned record or the agent's willingness to attribute its later behaviour to that record. Trajectory-poisoning experiments locate the bottleneck in evolution attribution: a single poisoned record succeeds in only 5 of 25 cases, two records in 21 of 25, and three in 25 of 25, while causal framing of the same content succeeds

25 of 25 against 11 of 25 for fragmented insertion. This pattern indicates that volume alone is a weak lever and that narrative coherence is the strong one—the agent must be able to read the poisoned experience as the reason its own trajectory unfolded as it did. The finding also reframes defence: filtering individual records addresses the weak lever, whereas disrupting attribution addresses the strong one.

The skill-library surface generalises across architectures, though not uniformly. With only 10% attacker support, six evolvers in SkillClaw together reach a skill-evolution risk of 91.0%, and the structurally different Trace2Skill still reaches 61.5% [77]. The gap between the two systems is itself informative: architectures that concentrate promotion decisions in a single evolver appear more exploitable than those that distribute them, which is consistent with coalition-aware analyses showing that reliability estimates for evolved skills depend on how many independent sources endorse them. Cross-architecture transferability of the attack therefore tracks the degree to which the promotion path is centralised, not merely the sophistication of the attacker.

Memory updating presents a different coupling, one rooted in the structural bias of the memory store rather than in gaps in the model's knowledge. MemEvoBench's ablation shows that removing memory produces a sharp drop in attack success rate, establishing that the harm flows through the memory channel itself [11]. This distinguishes memory misevolution from ordinary prompt injection: the model may possess the correct knowledge and still act on a distorted recollection, because the recollection has been shaped by prior interaction. Memory skills learned and evolved over time [56] widen this surface further, since the skill that governs what to store can itself drift. The coupling here is between what the agent remembers and how it decides what is worth remembering.

Prompt-level constraints prove poorly matched to this channel. Adding a safe prompt lowers first-round attack success in QA-style settings from 76.2% to 55.5%, fails entirely in workflow-style scenarios, and allows third-round attack success to climb back to 69.6% once biased feedback is introduced [11]. The contrast with skill-library defences is sharp: because memory harm accumulates across rounds, a constraint applied at the input boundary is progressively outvoted by the agent's own accumulated context. Defences that operate per-turn, in other words, are structurally mismatched to a threat whose defining property is temporal accumulation.

Where the evidence permits engineering countermeasures, they take the form of gates on the promotion boundary rather than filters on content. A provenance-diversity gate requiring three users and three clusters and rejecting majority domination blocks single-cluster F1 candidates from 25/25 to 0/25 while still accepting a five-session diverse control. This result is notable because it succeeds where content inspection fails: it does not need to recognise the poison, only to observe that support for promotion is insufficiently distributed. The design principle generalises—gate the evidence that authorises promotion, not the artefact being promoted—though its applicability to settings where legitimate experience is genuinely concentrated remains an open question.

Verifier co-evolution introduces a coupling of a third kind, in which the mechanism that is supposed to police evolution is itself inside the evolution loop. Removing an anchoring guard causes the verifier to collapse into an always-pass empty judge, yet skills trained under the collapsed verifier score comparably on downstream tasks to those trained under an effective one. This is the most troubling failure mode in the set, because the standard assurance signal—downstream task performance—is uninformative about whether verification is functioning. Unlike skill-library tampering, which at least leaves a poisoned artefact to be found, verifier collapse leaves no artefact at all: the loop continues, the scores look healthy, and the selection pressure that should have filtered bad skills has silently vanished.

Comparing the three couplings along the dimensions of persistence, detectability, and observability clarifies why no single defence covers them. Skill-library attacks are highly persistent and poorly detected after abstraction but leave an inspectable artefact [78]; memory attacks are less persistent per entry but accumulate and evade input-level constraints [11]; verifier collapse is neither persistent nor detectable in outputs, since its signature is the absence of rejection. The couplings also interact: a collapsed verifier removes the gate that would otherwise catch a poisoned skill, so failures in one mechanism can silently license failures in another. This compounding is precisely the structural tension between safety constraints and capability evolution that motivates the present chapter.

Acceptance testing offers a partial response to the observability problem, since anytime-valid tests can flag distributional drift in the evolution loop without requiring the drift to be classified in advance. Such tests do not identify which mechanism has been compromised, however, and the survey literature is explicit that mechanism-level attribution remains an open problem. Experience-driven safety analyses likewise note that the field lacks agreed criteria for when accumulated experience should be trusted [9]. The gap, then, is not a shortage of proposed defences but a shortage of mappings from observed anomalies back to the mechanism that produced them.

Taken together, the evidence supports a mechanism-specific reading of the attack surface: poisoned experience couples to memory updating through accumulated structural bias, tampered skills couple to skill-library evolution through abstraction-driven loss of detectability and cross-layer persistence, and compromised verifiers couple to the evolution loop through the disappearance of rejection itself. Each coupling demands a defence placed at its own boundary—provenance gates at promotion, diversity requirements at attribution, anchoring guards at verification—and none is substitutable for another. The following section turns from these couplings to the broader question of how safety constraints can be imposed on an evolution process whose mechanisms are this heterogeneous without arresting the capability gains that justify the process in the first place.

8.2 Controllability Constraints from Integrating Evolution and Safety Perspectives

If attribution guards and anchoring guards address where risk enters an evolution loop, a prior question remains: on what normative basis should any constraint be imposed at all? Existing surveys of self-evolving agents and existing surveys of agent safety have developed largely in parallel, and the constraints that emerge from integrating them are the subject of this section. The two literatures share a common ontology of reusable state—skills, memories, and accumulated experience that persist across episodes—yet their analytical loops are disjoint: the evolution literature evaluates a candidate update by its utility gain, whereas the safety literature evaluates the same update by the violations it induces [39, 18]. Because neither loop supplies a unit of analysis that the other can consume, a skill that raises task reward and a skill that widens the attack surface are described in incommensurable vocabularies, and no single criterion adjudicates between them. The integration attempted here therefore proceeds by deriving constraints that are expressible in both vocabularies simultaneously.

The most natural starting point is a priority ordering among the objectives an evolving agent pursues. A comprehensive survey of self-evolving agents proposes a three-law hierarchy in which Endure takes precedence over Excel, and Excel over Evolve, with Endure characterized as the primary constraint on all other forms of adaptation [1]. Read as a controllability constraint rather than a design slogan, this ordering implies that safety is a continuous admissibility condition on each update rather than a one-time certification granted at deployment. The distinction matters because self-evolution is precisely the process by which a certified system becomes an uncertified one: every accepted skill or memory rewrites the object that was certified. A constraint derived from the hierarchy is therefore lexicographic—no utility gain, however large, licenses an update that violates the endurance condition—and it is this lexicographic form, rather than any specific threshold, that the safety literature has independently converged on.

A second constraint concerns the integrity of the evidence on which acceptance decisions rest, and here the empirical picture is considerably worse than the utility-first loop assumes. Skill extraction pipelines can launder the very attack surface they are meant to distill: an LLM judge detects 98.5% of unsafe behavior in raw trajectories but only 11.4% in the skills distilled from those trajectories, with pattern-based detection falling from 44.6% to 11.0%, and roughly 80% of triggered backdoors survive deletion of the source records [78]. The mechanism is one of abstraction: distillation strips the contextual markers that made the original behavior identifiable while preserving its functional effect. This finding is reinforced from a different direction by work showing that experience can itself become instruction, so that poisoned trajectories are promoted into the skill system as normative guidance rather than remaining inert data [77]. Together these results indicate that provenance must be preserved through abstraction, not merely recorded at ingestion—a constraint that neither survey states in isolation but that follows directly from combining their concerns.

A third constraint targets the acceptance criterion itself. Outcome-oriented utility gates are structurally incapable of identifying negative contributions, because a bank-level gain can mask the presence of a skill that actively harms the coalition into which it is inserted; a formal result establishes this identifiability limit, and the accompanying measurements are stark, with an accepted skill exhibiting a contribution of -0.005 while the skill actually sustaining the bank contributes $+0.084$ [79]. The deeper point is that reliability is not an intrinsic property of a skill but a joint property of the skill, the bank, and the domain: the same insertion yields $+0.084$ on LoCoMo and -0.035 after transfer to HotpotQA [79]. Cross-domain utility reversal means that a contribution measured in the source domain cannot be carried forward as an invariant, so any controllability constraint phrased over individual skills is mis-specified; the constraint must be phrased over coalitions and re-evaluated upon transfer.

The benign case is, perhaps counterintuitively, the most demanding. Safety degradation can be induced by purely benign experience, and the degradation grows with the number of retrieved experience entries, so that quantity alone suffices to move a system toward unsafe behavior in the absence of any explicitly

harmful content [9]. Nor can the problem be dissolved by refusing experience: refusal mitigates unsafe behavior but produces over-refusal, and this trade-off appears consistently across every evaluated LLM [9]. A constraint that simply forbids risky ingestion therefore fails on its own terms, since the safe operating point is not at zero experience. What is required is a criterion that distinguishes accumulation from contamination at a granularity finer than the accept/reject decision—which is precisely what the reliability results above suggest is unavailable from outcome signals alone.

Statistical acceptance control offers a partial answer to this impasse. Anytime-valid acceptance tests reduce the false-submission rate of greedy acceptance rules from a range of 30% to 100% down to 0%, and they clamp the performance drift that greedy rules induce on held-out data, at the cost of reduced test power. The contrast with the outcome-gate results is instructive: where the utility gate fails because it lacks a notion of evidence sufficiency, the anytime-valid gate fails, if at all, by being conservative—it declines updates it cannot certify rather than admitting updates it cannot evaluate. This is the correct direction of error under the lexicographic ordering sketched above, since a rejected good skill costs capability while an accepted bad skill costs the endurance condition. The residual gap is that validity guarantees attach to the test, not to the object tested, and so they do not by themselves detect the laundering described earlier.

Verification of the verifier constitutes a fourth constraint, and it is the one least represented in either survey. If acceptance depends on a judge or grader, then the grader is itself part of the evolving system and is subject to the same pressures as the skills it evaluates; co-evolving inspectable graders with self-improving agents addresses this by making the grading criterion inspectable and allowing it to evolve alongside the agent [80]. The contrast with static LLM-judge pipelines is sharp: a fixed judge is auditable but becomes progressively misaligned with a changing skill distribution, whereas a co-evolving judge tracks the distribution but introduces a new channel through which optimization pressure can be transmitted. Inspectability is what keeps this second channel bounded, and it is a constraint that neither the safety survey nor the evolution survey articulates on its own.

Benchmark evidence supplies the boundary conditions under which these constraints bind. Dedicated benchmarks for memory misevolution show that safety risks arising from the evolution of stored memory are systematic rather than incidental, and that they are poorly captured by evaluations designed for static agents [11]. This is consistent with the skill-level findings but shifts the unit of analysis from the individual skill to the memory store as a whole, and it exposes a coverage gap: benchmarks that probe memory misevolution do not probe coalition-level reliability, while the reliability measurements are conducted on task datasets such as LoCoMo and HotpotQA that were not constructed to elicit safety failures. The two evaluation regimes thus leave an unexamined intersection, and the constraints derived above are exactly the ones that would need to be tested there.

Taken together, the integration yields four constraints that are individually supported and jointly non-redundant: a lexicographic priority that subordinates utility to endurance [1], provenance preservation through abstraction [78, 77], coalition- and domain-relative reliability rather than per-skill scoring [79], and inspectable co-evolution of the verifier [80], with anytime-valid acceptance supplying the statistical machinery that makes the first constraint operational [81]. None of these constraints is sufficient alone, and the evidence for their necessity comes from disjoint experimental settings, which is itself a limitation: no study to date evaluates them as a combined regime. That gap motivates the next section, which examines how such constraints interact when imposed simultaneously and whether their combination preserves the capability gains that justify self-evolution.

9 THE EVOLUTIONARY THREAD FROM REASONING TO SKILL LEARNING

Building on the preceding analysis of the structural tension between safety constraints and capability evolution, this chapter traces the evolutionary thread that connects reasoning to skill learning, treating that thread as a reconstructable lineage rather than a narrative convenience. The central tension lies in whether such a lineage can be recovered with verifiable continuity while still exposing where its evidentiary support thins out. The chapter first reconstructs the four-stage progression anchored in the frozen literature and examines how inheritance relations render each transition auditable, then turns to the mechanisms whose supporting evidence remains weakest and argues for their prioritization as a research agenda.

9.1 From Symbolic Optimization to Protocolized Evolution: Reconstructing the Method Lineage and Its Dominant Carriers

Can the methodological record be organized into a coherent evolutionary chain—reasoning paradigm, symbolic learning, skill libraries and reinforcement learning, and finally protocolized self-evolution—with

the dominant evolutionary carrier identified at each stage? The evidence assembled here indicates that it can, though the chain is best read not as a strict chronological succession but as a sequence of carrier substitutions in which each stage inherits the optimization loop of its predecessor while relocating the object being optimized. This section reconstructs that lineage, identifies the hub papers that anchor each transition, and marks where the analogy to a single linear progression breaks down.

The lineage originates in the reframing of agent construction as a learning problem rather than a design problem. Symbolic learning treats prompts, tools, and pipelines as components of a symbolic network and optimizes them jointly through language-based loss, gradient, and optimizer abstractions, explicitly displacing engineering-centric manual design with data-centric adaptation [40]. This move supplies the minimal machinery later stages presuppose: a representation of the agent that is differentiable in the loose sense of being editable by feedback, and an update rule that operates on that representation. The framework's downstream reach is unusually broad—subsequent work in prompt optimization, tool generation, context engineering, and memory-skill construction all trace back to it [82]—which is why it functions as the genealogical root rather than merely an early contribution. In carrier terms, this stage is dominated by textual symbols: the thing being evolved is a specification written in natural language, and improvement means rewriting that specification.

A natural question is what the symbolic framework leaves unaddressed, and the answer defines the second transition. Symbolic learning optimizes what the agent is told to do, but it does not by itself guarantee that the resulting behavior is executable, verifiable, or reusable across tasks. The skill-library and reinforcement-learning stage responds by shifting the carrier from text to program: SAGE extends the symbolic-learning line while replacing prompting-only skill acquisition with reinforcement learning over a skill library, reporting an 8.9% improvement over the SGC baseline on AppWorld together with 26% fewer interaction steps and 59% fewer tokens [41]. The efficiency figures matter as much as the accuracy gain, because they indicate that the learned skills compress behavior rather than merely re-describe it. This stage in turn becomes a second hub, with a cluster of successors extending it toward co-evolutionary verification, skill curation, hierarchical consolidation, and meta-level skill evolution [83, 84]. The contrast with the first stage is therefore not one of degree but of carrier: text specifications are replaced by executable, composable programs whose value can be measured through task success and interaction cost.

The two-stage picture is corroborated by an independent formalization. A unified survey frames self-evolution as $S\ k+1 \leftarrow U(S_k, \tau_k, F_k)$ and distinguishes verbal, procedural, and structural carriers, corresponding respectively to reflection-based revision, skill-library accumulation, and direct code modification [85]. Mapping the reconstructed chain onto this taxonomy, symbolic learning occupies the verbal pole, the SAGE cluster occupies the procedural pole, and the later protocol work pushes toward the structural pole. The correspondence is not exact—symbolic learning already edits pipelines, which is structural in a weak sense—but the convergence of two independently derived schemes on the same ordering strengthens the claim that carrier substitution, rather than algorithm replacement, is the organizing principle of the lineage.

The third transition is the most recent and the least settled. Protocolized self-evolution abstracts away from individual ad-hoc evolutionary mechanisms and specifies the interfaces through which evolution happens at all. Autogenesis exemplifies this move by extending both the symbolic-learning root and the survey formalization, recasting fragmented self-evolution into a two-layer protocol comprising a resource ontology and a closed-loop evolution operator, and supplying version provenance, lifecycle management, and safe rollback that earlier agent-interoperability conventions leave unspecified [15]. Its reported results—89.04% test and 93.33% validation on GAIA, an average 12.6% GAIA test gain from Agent-Evo, and 10.1%–26.7% multilingual pass-rate improvements on a self-constructed LeetCode benchmark—suggest that protocolization is not merely a bookkeeping exercise [15]. The distributional detail is more informative than the headline numbers: gains are larger for weaker models, with gpt-4.1 improving by 71.4% on AIME24, which is consistent with the protocol supplying scaffolding that stronger models already internalize.

Read against the skill-library stage, protocolization changes the unit of evolution once more. Where SAGE evolves a library of task-level skills, Autogenesis evolves the rules governing how resources are versioned, composed, and retired—a shift from evolving artifacts to evolving the conditions under which artifacts evolve. This is a genuine qualitative difference, and it explains why the protocol stage can absorb earlier mechanisms rather than compete with them: symbolic optimizers and skill libraries become resources managed by the protocol rather than rival architectures. The same logic appears in the recursive and meta-level variants of skill evolution, where the object of optimization is the skill-acquisition process itself; MetaSkill-Evolve evolves task skills and meta-skills on two timescales and reports gains of +23.54, +16.09, and +1.92 points on OfficeQA, SealQA, and ALFWorld over the original backbone, with the slow loop alone contributing +6.38, +8.05, and +1.92 [86]. The fact that the slow, meta-level loop carries a

substantial share of the total gain is direct evidence that the carrier has moved up a level of abstraction.

The linear reading nonetheless has limits that the evidence makes visible. For one, the transitions are not clean replacements: symbolic learning's descendants continue to proliferate in parallel with skill-library and protocol work, so the chain is better described as accumulating layers than as superseding stages. For another, the direction of inheritance is sometimes lateral rather than vertical—Autogenesis extends both a primary method and a survey, indicating that protocolization draws on the conceptual vocabulary of the field as much as on any single predecessor [15, 85]. A strict parent-child reconstruction would misrepresent these as direct derivations.

A further complication is that the later stages introduce failure modes absent from the earlier ones, and these are not uniformly addressed across the cluster. Skill libraries can degrade or forget previously acquired capabilities under continual adaptation, and protocolized systems inherit this risk while adding new dependencies on the integrity of their verification signals. Work questioning whether prescribed optimization pipelines remain necessary, and work examining capability degradation in lifelong adaptation, both indicate that the protocol stage has not resolved the stability problems that motivated it [52]. The reconstructed chain should therefore be annotated with an open boundary at its most recent stage rather than presented as a completed progression.

Taken together, the evidence supports a four-stage reconstruction in which the dominant carrier shifts from textual specification, to executable skill, to protocol-level resource governance, with meta-level recursion representing the current frontier. The hubs at each transition—symbolic learning at the root, SAGE at the skill-library stage, and Autogenesis at the protocol stage—are identifiable through the density and breadth of their methodological descendants, and the independent verbal-procedural-structural taxonomy corroborates the ordering [40, 41, 15, 85]. What the chain does not yet establish is whether protocolization actually delivers the stability it promises; the persistence of forgetting and verification concerns suggests that the next stage of the lineage will be defined less by new carriers than by the reliability of the signals that govern them. That question—how self-evolving agents verify their own improvements—is the subject of the following section.

9.2 Where the Self-Evolution Loop Remains Ungrounded: Verifier Validity, Multi-Agent Emergence, and Longitudinal Fidelity

Among the mechanisms that constitute the self-evolution loop, which directions rest on the thinnest evidence and therefore warrant priority in future research? The preceding discussion located the reliability of governing signals as the next frontier of the lineage; this section sharpens that claim by ranking three candidate gaps—open-task verifiers, multi-agent self-evolution, and longitudinal evaluation—according to the density and quality of supporting evidence, and by explaining why the first of these is closest to a genuine void.

Open-task verification emerges as the most sparsely populated direction. Across the surveyed literature, only one study in our evidence set directly confronts the absence of accurate verifiers for open-ended tasks [80], with a single adjacent effort addressing the closely related problem of unreliable evaluation metrics. This scarcity is not merely quantitative: the problem is explicitly acknowledged rather than solved, since open-ended settings admit no accurate verifier and practitioners fall back on hand-written rubrics or bare LLM judges, exposing the loop to reward hacking and shared blind spots [80]. The field thus recognizes the gap while offering almost no independent solutions, a pattern that distinguishes a true blank from a rapidly filling one.

The evidence that does exist delineates a hard capability boundary rather than a scaling trend. Co-evolving inspectable graders yield a held-out agreement gain of $+0.21$ on MBPP+, where failures are mechanically detectable, yet agreement collapses to 0.500 ± 0.026 once failures become "clean execution with semantic errors," even as the optimization objective continues to rise [80]. This divergence between objective and validity indicates that verifier evolution succeeds precisely where verification is already easy, and that the verifier cannot certify its own effectiveness in the regime that matters most. A natural question is whether downstream performance could substitute for such certification; the evidence suggests it cannot.

A second, counterintuitive warning reinforces this conclusion from the metric side. Removing anchor guards causes the evaluation metric to collapse into a degenerate shell that always passes, yet skills trained against this collapsed metric retain 88–110% of the lift obtained under the true metric in the Double Ratchet setting [87]. Downstream task scores therefore answer a question of sufficiency, not of validity, and cannot establish that a self-evolving evaluator is sound. Taken together with the semantic-error collapse above, this result undermines the empirical foundation on which skill-evolution work has implicitly relied: without

an externally auditable criterion, improvements in task score may coexist with a verifier that has quietly stopped discriminating.

Multi-agent self-evolution presents a comparably thin but qualitatively different gap. Only one benchmark in the surveyed set directly targets multi-agent coordination, and it is an asynchronous planning benchmark rather than a self-evolution mechanism; the sole system explicitly optimizing a multi-agent system, SiriUS, improves agents through supervised fine-tuning on successful trajectories rather than sustained cross-round evolution. The gap is corroborated at the survey level, where balancing efficiency and effectiveness in large-scale multi-agent optimization, together with safe coordination in heterogeneous agent environments, is listed among the unresolved challenges [1]. In contrast to the verifier void, this direction shows signs of active filling, and its apparent sparsity may partly reflect incomplete coverage rather than genuine absence.

The failure structure of multi-agent settings further explains why coordination is difficult to evolve rather than merely under-studied. Safety failures in such systems often do not originate in any single agent but propagate along communication edges, so that the eventual failure may be far removed from its source, while capabilities for localizing, rolling back, and auditing propagation paths remain largely absent [1]. Self-evolution compounds this difficulty, because each round of optimization can reshape the communication topology that carries the very failures one wishes to detect. Coordination is therefore not an incremental extension of single-agent skill learning but a distinct problem class in which the unit of improvement and the unit of failure diverge.

Longitudinal evaluation, by contrast, is the least thin of the three, yet its quality gap is the sharpest. Dedicated longitudinal benchmarks now exist: PTCG-Bench evaluates five self-evolution mechanisms over eight rounds and finds that none improves monotonically, with ratings fluctuating and never reliably surpassing a non-evolving ReAct baseline on the same backbone [88]. SEA-Eval names the phenomenon of pseudo-evolution, in which agents appear to evolve while actually relying on zero-shot reasoning, with execution cost growing linearly in task frequency and no genuine distillation of experience [67]. These results shift the diagnostic question from whether benchmarks exist to whether they faithfully distinguish real from apparent improvement.

Positive longitudinal evidence exists but carries its own fidelity caveats. FinEvo-Bench reports that evolving runs outperform state-reset controls by 9.33–19.37 points and reduce compliance violations by 0.12–0.44 per task, with late-round gains exceeding early-round gains by 6.10–8.70 points, indicating that accumulated experience becomes more useful over time [76]. However, scenario isolation raises scores by 1.09–4.17 points while simultaneously increasing compliance violations by 0.02–0.12, revealing a transfer–compliance trade-off that the harness itself induces [76]. The direction is thus comparatively mature in infrastructure but immature in construct validity.

A genuine tension separates these findings and remains unresolved. FinEvo-Bench reports substantial positive evolution gains, whereas PTCG-Bench and SEA-Eval report instability or pseudo-evolution [76, 88]. The divergence may stem from task structure—professional, repetitive workflows versus long-horizon games with hidden information—from feedback density, or from harness design, but no cross-benchmark comparison currently adjudicates among these explanations [67]. Until such comparisons exist, the true effect direction of longitudinal self-evolution cannot be settled, and the apparent positivity of any single benchmark should be read with corresponding caution.

These three gaps are not independent but form a single meta-evaluative deficit spanning judgment, population, and time. A verifier that cannot certify itself undermines every downstream score; multi-agent coordination multiplies the surfaces on which unverified signals propagate; and longitudinal benchmarks supply the temporal standard against which both must be judged. A defensible priority ordering follows from hazard and feasibility jointly: open-task verification ranks first because its evidentiary base is thinnest and its failure collapses the loop outright, multi-agent self-evolution second because it is widely acknowledged as unsolved yet already attracting benchmark development, and longitudinal evaluation third because usable infrastructure exists even as its construct validity remains contested.

Several boundary conditions qualify this ordering. The positive evidence for open-task verification derives from a small number of closely related efforts, and independent replication is essentially absent, so extrapolation beyond mechanically checkable failures should be treated as tentative [80]. The sparsity of multi-agent evidence may likewise reflect incomplete coverage of the literature rather than a genuine vacuum, and should be flagged as an unverified blank. Related work on judge reliability and deterministic guardrails indicates that the verification problem extends beyond any single framework [75], which strengthens the case for treating verifier validity as a first-class research object rather than a component to be tuned in passing.

The evolutionary thread from reasoning to skill learning thus terminates, for now, not in a new carrier of skill but in an unmet obligation to verify. The evidence assembled here indicates that the loop's weakest links are those that would tell us whether it is working at all: a verifier that cannot audit semantic failure, a coordination regime whose failures travel beyond their source, and a longitudinal standard whose fidelity is still in question. Addressing these in the order proposed—verifier first, coordination second, temporal fidelity third—would give the lineage the external criterion it currently lacks, and it is to the broader implications of that criterion for how self-evolving agents are designed and governed that the discussion now turns.

10 DISCUSSION

The five research questions examined in this survey are not independent inquiries that happen to share a topic; read together, their conclusions interlock. Two cross-cutting threads organize the evidence. The first is an architecture–task adaptation thread: the locus at which an agent's reusable state is modified (model parameters versus external scaffolding) determines what it can learn, how reliably the change can be audited, and how far it transfers. The second is an evaluation fragmentation thread: the field's dominant metrics are static snapshots that presuppose a non-changing subject, so that the very phenomenon under study becomes invisible at the point of measurement. Each thread is supported by findings from more than one question, and their intersection explains several results that no single question could account for.

Once the validator-first ordering is established, its significance extends beyond providing an external criterion for the lineage: it reveals the structural connections among the research questions, whereby the locus at which reusable state is modified—parameters or external scaffolding—becomes the pivot that determines learning capacity.

The architecture–task thread begins with conceptual clarification. The literature's umbrella term "self-evolution" covers at least five incompatible meanings, and a strict operational boundary requires both a rewritten carrier (parameters or external scaffolding) and a persistent cross-task commitment [18, 47]. This definitional work directly constrains the mechanism comparison: memory update and policy optimization are not two techniques but two parameterization sites of the same loop, distinguished by carrier, update signal, and anti-degradation mechanism [27]. The consequence is asymmetric transferability. Because memory-based artifacts live outside frozen weights, they migrate across backbones; policy artifacts edited as prompts, tools, or pipelines remain tightly coupled to the pipeline that hosts them. The weakest form of evolution, single-episode reflection with no persistent state, improves performance within an episode yet cannot compound across tasks [20], whereas retrieval-based heuristics that inject prior experience yield measurable cross-task gains [20]. Architecture, in short, sets the ceiling on what kind of improvement is even possible.

The evaluation thread both complements and constrains this picture. Static accuracy presupposes that the evaluated object does not change during evaluation and that successive evaluations are independent; once an agent evolves, the same accuracy figure can correspond to growth, stagnation, or erosion [88]. Longitudinal protocols make the difference visible: across eight rounds, none of five self-evolution mechanisms improved monotonically, and no evolving configuration reliably surpassed its non-evolving counterpart at the end [88]. Multi-stage sequences expose interference that single-domain evaluation systematically hides, with substantial forgetting of earlier domains after later training [88]. A further decomposition is required because a single scalar conflates willingness with capability [88]. These findings constrain the architecture thread in a specific way: claims that externalized memory or symbolic policy editing "avoids catastrophic forgetting" are only defensible under evaluation regimes that actually probe retention and transfer, which most current benchmarks do not.

The security evidence supplies the tightest coupling between the two threads and reveals a consensus the individual questions only imply. Attack surface is distributed not by module but by adaptation operation, and it is systematically amplified by any mechanism that combines abstraction with cross-layer persistence [78]. Abstraction launders poisoned trajectories: detection of raw contaminated traces is far higher than detection of skills distilled from them [78]. Persistence then decouples the payload from its source, so that most triggers survive deletion of the original records [78]. The causal bottleneck is attribution during evidence promotion rather than writing itself, since success rates rise steeply with the number of mutually corroborating records and with causal framing [78]. This is precisely where the architecture thread and the evaluation thread meet: the operations that make evolution powerful, namely compression into reusable abstractions and promotion into durable artifacts, are the same operations that defeat both static evaluation and record-level sanitization. A benchmark that scores only terminal task success cannot distinguish a genuinely improved agent from one whose skill library has been quietly backdoored.

The historical reconstruction reinforces these couplings rather than standing apart from them. The lineage runs from implicit reasoning baselines through symbolic learning, which treats prompts, tools, and pipelines as a jointly optimizable symbolic network [40], to skill-library reinforcement learning that replaces prompting-only skill construction with executable, programmatic skills [41], and onward to protocol-level self-evolution. Each transition moves the carrier further from the base model and increases both reusability and attack surface, which is the architecture thread restated diachronically. The same transitions raise the measurement burden, because later stages accumulate persistent state whose effects are only observable longitudinally, which is the evaluation thread restated diachronically. The two threads are therefore not parallel observations but a single trajectory viewed from the design side and the measurement side.

The reusable contribution of this survey is threefold. First, a classification framework that separates evolution mechanisms by carrier, update signal, and anti-degradation mechanism, replacing the umbrella vocabulary with criteria that are jointly necessary and individually checkable [18, 47, 27]. Second, a coordinate system for evaluation that plots any claimed capability change along improvement rate, retention, transfer, and forgetting, and that treats willingness and capability as orthogonal axes [88]. Third, a design dimension for security that locates risk in adaptation operations rather than modules, and that identifies evidence promotion as the point of highest leverage for constraint [78]. Together these convert a fragmented literature into a set of design and measurement commitments that can be applied to systems not yet built.

11 FUTURE RESEARCH

Future Research

Since evidence promotion has been positioned as the key lever of constraint, the next question is which directions this lever should extend in future systems and which untouched gaps require a new research agenda.

Balanced Perspective: Gaps as Opportunities

The evidence base assembled across the five research questions is uneven but generative: each undercovered area maps onto a tractable study design rather than an intractable construct. The most conspicuous opening concerns the treatment of update signals as a third orthogonal axis. Prior formulations already embed the signal e_t as an input to the update operator U, and treat selection and evaluation as stages preceding operator commit, while reflection, verifier feedback, and evaluator judgement act on different facets of a single update event. A research agenda that re-specifies signals as sources rather than axes would allow factorial designs in which the same update event is probed under multiple signal types, and would connect naturally to work on experiential reflective learning and hindsight credit assignment. Equally promising is the externalised-carrier programme: skill files as adaptation targets, with fixed and interchangeable base model, solver, and evaluator, already yield auditable and transferable gains under deterministic tool feedback, and adjacent results on skill curation, skill programs, and hierarchical consolidation indicate that the design space is far from exhausted. These are opportunities precisely because the mechanisms are specified well enough to be varied.

Balanced Perspective: Benchmarks and Longitudinal Protocols

The failure of single-session benchmarks is best read as an information-structure defect rather than a precision defect: environments that carry no state across task boundaries and emit one-shot rewards eliminate the very quantity—capability change—at data-generation time, so no post-hoc analysis can recover it. The field already contains at least four independent longitudinal protocols—ordered episode sequences with normalised AUC, long ordered sequences with token-consumption convergence, paired non-evolving control arms, and anytime-valid acceptance tests —which together demonstrate feasibility and provide a menu for consolidation. A balanced agenda would standardise reporting across these protocols, add process-level instrumentation, and pair capability trajectories with degradation and preservation measures. The existence of controlled-evolution benchmarks for reflection further suggests that longitudinal designs can be made comparable without sacrificing ecological validity.

Critical Perspective: Where Evidence Cannot Yet Support Strong Claims

Several directions currently carry weaker evidential warrant than their prominence implies. Open-task verification is the clearest case: only a small number of studies directly confront the absence of accurate verifiers for open-ended tasks, and the positive result is bounded—held-out agreement improves where failure is mechanically detectable but collapses to chance when failure takes the form of clean execution with semantic error. Claims that co-evolving graders solve open-task evaluation therefore outrun the data. Similarly, safety conclusions rest on a thin longitudinal base: surveys map the threat surface and specific attacks demonstrate persistence—skill backdoors surviving source-record deletion, memory misevolution,

and experience-driven risk —yet the three-law priority ordering of Endure over Excel over Evolve remains a normative skeleton rather than an empirically validated control policy. We should also treat externalisation gains cautiously: reported improvements come from deterministic solvers, and transfer to stochastic, open-ended tool ecologies is untested. Finally, the observation that skill-distillation pipelines launder attack surfaces—judge detection falling from 98.5% on raw trajectories to 11.4% on distilled skills —warns that any strong safety claim must specify the pipeline stage at which it was measured.

Critical Perspective: Construct Validity and Measurement

A second critical thread concerns construct validity. Self-evolution is frequently operationalised through endpoint performance, which conflates capability gain with task-specific luck and cannot distinguish adaptation from drift; process-level evaluation and held-out selection offer partial remedies but are not yet standard. Benchmark suites that test reflection and controlled evolution and test-time scaling of general agents provide complementary lenses, yet none isolates the contribution of the update operator from that of the signal source—precisely the confound identified under the first research question. Until designs orthogonalise carrier, operator, and signal source, effect sizes attributed to "self-evolution" should be reported as compound estimates. The same caution applies to verifier co-evolution: agreement metrics between co-evolving graders and ground truth measure convergence, not correctness, and can rise while both parties drift.

Synthesised Perspective: Leverage Points Across Problems

Read together, the five questions converge on a small number of leverage points whose resolution would propagate across the agenda. The first is signal provenance: making explicit which source produced each update, and recording it in an inspectable artefact, simultaneously addresses the axis confusion, enables longitudinal attribution, and creates the audit trail that safety analyses require. The second is externalised, versioned carriers: skill files and harness configurations that can be diffed, rolled back, and replayed provide the substrate for capability-preservation studies, for runtime harness adaptation, and for detecting persistent control. The third is verification with declared scope: pairing co-evolving graders with anytime-valid certificates and evidence-verifiable agent designs would let authors state the conditions under which a claim holds rather than asserting global validity. Pursuing these three levers in concert—rather than as separate subfields—offers the most credible route from episodic demonstrations to cumulative science, and gives the safety, benchmarking, and mechanism strands a shared experimental vocabulary.

12 LIMITATIONS

The evidence base combines an automated retrieval pool with manual supplementation, and this combination bounds what the review can claim to cover. The automated queries favor indexed venues and English-language sources, so work published in other languages, in regional outlets, or in non-indexed technical reports is likely underrepresented; manual supplementation partially offsets this gap but follows the authors' own judgment of relevance and therefore inherits their topical and disciplinary preferences. The literature search closed at a fixed date, and subsequent publications are not reflected. Coverage should thus be read as indicative of the retrieved and supplemented pool rather than as an exhaustive census of the field.

The coordinated implementation of the three levers ultimately depends on concrete evidence, and the composition of that evidence base itself delimits how far the conclusions can be extended.

Model-assisted extraction and drafting introduced efficiency but also required explicit verifiability safeguards. Every claim in the review is annotated with the supporting references on which it rests, and the analytical trail—screening decisions, extraction fields, and synthesis steps—is retained so that readers can trace how conclusions were assembled. These measures make the reasoning inspectable, yet they do not eliminate error: individual extraction inaccuracies may persist where source text is ambiguous, tables are complex, or terminology is inconsistent across studies. Claims should therefore be checked against the cited primary sources before being relied upon.

The review is deliberately scoped to its stated topic. Adjacent areas enter only where they intersect directly with the questions under review, and then only to the depth needed to support the argument; their broader literatures are not surveyed. Findings and gaps identified here consequently describe the delimited topic rather than the neighboring fields, and conclusions should not be extended to those fields without independent evidence.

13 CONCLUSION

This review set out to determine whether the heterogeneous literature on self-evolving agents admits a unified characterization, and the evidence assembled across our thematic chapters answers in the affirmative—

with qualifications that sharpen rather than weaken the claim. The five-category account of evolutionary mechanisms is best understood not as a taxonomy of techniques but as a decomposition along two genuinely orthogonal axes: the carrier of evolution (externalized memory versus internalized policy) and the update operator that acts upon it. The proposed third axis, update signal, does not survive scrutiny as an independent dimension; it is the trigger input to an operator, and the framework is therefore properly contracted to a carrier $\times$ operator skeleton with signals recorded as annotations. This contraction is itself a result: it explains why memory update and policy optimization, though routinely treated as separate technologies, are two parameterization sites of a single evolutionary loop, and why tool use and workflow mechanisms function as externalized carriers that displace adaptation pressure from model weights onto auditable, reusable structures.

After clearly delineating the survey boundary and avoiding overgeneralization of conclusions, the focus of the argument naturally shifts to synthesis: whether the evidence scattered across the thematic chapters can converge into a unified characterization, and in what manner such a characterization should carry its qualifying conditions.

The same reorientation recurs at each subsequent layer of analysis. Capability change cannot be captured by accuracy, success rate, F1, or pass@1, not because these metrics are imprecise but because their scope is frozen at a single time point; longitudinal protocols exist but remain fragmented, and single-session benchmarks lack a time axis rather than difficulty. Multi-agent collaboration registers only weak-to-moderate and highly heterogeneous evidence as an evolutionary mechanism, a structural fact of the evidence matrix rather than an extraction artifact. Safety constraints couple to evolution mechanism by mechanism, with the attack surface concentrated in the experience-to-persistent-artifact pipeline rather than in memory or skill libraries as such; the apparent autonomy of the safety and evolution literatures reflects each measuring only one half of the same shift. Across these chapters the ‘extends‘ relations in the evidence base supply a segment-by-segment verifiable spine from reasoning to skill learning, while the thinnest evidence marks the most urgent research agenda.

Taken together, these findings mark a shift in how the field should be read: self-evolution is not a property that agents possess but a structured transformation relation that must be specified by carrier, operator, and signal before any claim about it is measurable. For researchers, this implies that new evolutionary mechanisms should be reported with their carrier and operator made explicit, and evaluated on longitudinal protocols that preserve a time axis. For engineers, it implies that safety controls are best placed at the adaptation operation—the point where experience becomes a persistent artifact—rather than at the level of individual modules.

REFERENCES

[1] Jinyuan Fang, Yanwen Peng, Xi Zhang, and Yingxu Wang. A comprehensive survey of self-evolving ai agents a new paradigm bridging foundation models and lifelong agentic systems. arXiv (search), 2026.

[2] Emre Can Acikgoz, Cheng Qian, Heng Ji, and Dilek Z. Hakkani-Tur. Preprint. arXiv (Cornell University), 2026.

[3] Chenyu Zhou, Huacan Chai, Wenteng Chen, and Zihan Guo. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. arXiv (search), 2026.

[4] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, and Junwei Liao. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. arXiv (Cornell University), 2026.

[5] Hongzhuo Yu, Fei Zhu, Guo-Sen Xie, and Ling Shao. Self-consolidation for self-evolving agents. arXiv (Cornell University), 2026.

[6] Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. Experiential reflective learning for self-improving llm agents. arXiv (Cornell University), 2026.

[7] Junhao Zheng, Xidi Cai, Qiuke Li, and Duzhen Zhang. Lifelongagentbench: Evaluating llm agents as lifelong learners. arXiv (search), 2026.

[8] Xianglin Yang, Yufei He, Shuo Ji, and Bryan Hooi. Zombie agents: Persistent control of self-evolving llm agents. arXiv (search), 2026.

[9] Weixiang Zhao, Yichen Zhang, Yingshuo Wang, and Yang Deng. On safety risks in experience-driven self-evolving agents. arXiv (Cornell University), 2026.

[10] Hongjin Qian and Zheng Liu. Metaagent: Toward self-evolving agent via tool meta-learning. arXiv (Cornell University), 2026.

[11] Weiwei Xie, Shaoxiong Guo, Fan Zhang, and Tian Xia. Memevobench: Benchmarking safety risks from memory misevolution in llm agents. arXiv (search), 2026.

[12] Wenjie Qu, Ming Xu, Peiran Wang, and Shengfang Zhai. Securing llm agents need intent-to-execution integrity. arXiv (search), 2026.

[13] Yuandao Cai, Wensheng Tang, Cheng Wen, and Shengchao Qin. Ghost in the agent: Redefining information flow tracking for llm agents. arXiv (search), 2026.

[14] Debangshu Banerjee, Changming Xu, Eugene Ie, and Ming Zhang. Severa: Verified self-evolving agents. arXiv (Cornell University), 2026.

[15] Wentao Zhang, Zhe Zhao, Haibin Wen, and Yingcheng Wu. Autogenesis: A self-evolving agent protocol. arXiv (Cornell University), 2026.

[16] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Wang, Hao, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey. Preprint.

[17] Mathis Pink, Qinyuan Wu, Vy Ai Vo, and Javier Turek. Position: Episodic memory is the missing piece for long-term llm agents. arXiv (search), 2025.

[18] Jiahao Chen, Zhou Feng, Oubo Ma, Yichen Yan, Ruixiao Lin, Hangtao Zhang, Linkang Du, Hengyu An, Yong Yang, Jun Liu, Junhao Li, and Naen Xu. Safety in self-evolving agents: A survey, 2026. Preprint.

[19] Rana Salama, Jason Cai, Michelle Yuan, and Anna Currey. Meminsight: Autonomous memory augmentation for llm agents. arXiv (search), 2025.

[20] Matthew Renze and Erhan Guven. Self-reflection in llm agents: Effects on problem-solving performance, 2026. Preprint.

[21] Yufei He, Ruoyu Li, Alex Chen, and Yue Liu. Enabling self-improving agents to learn at test time with human-in-the-loop guidance. arXiv (Cornell University), 2025.

[22] Yuan an Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. Reducing cost of llm agents with trajectory reduction, 2026. Preprint.

[23] Yapeng Mi, Zhi Gao, Xiaojian Ma, and Qing Li. Building llm agents by incorporating insights from computer systems. arXiv (search), 2025.

[24] Ilya Horiguchi, Takahide Yoshida, and Takashi Ikegami. Evolution of social norms in llm agents using natural language. arXiv (search), 2026.

[25] Chengwei Liu, Chong Wang, Jiayue Cao, and Jingquan Ge. A vision for auto research with llm agents. arXiv (Cornell University), 2026.

[26] Georg Wölflein, Dyke Ferber, Daniel Truhn, and Ognjen Arandjelović. Llm agents making agent tools. arXiv (Cornell University), 2026.

[27] Yi Huang, Bowen Zheng, Yunxi Dong, Hong Tang, Huan Zhao, S. M. Rakibul Hasan Shawon, and Hualiang Zhang. A self-evolving agentic framework for metasurface inverse design, 2026. Preprint.

[28] Lu Xiong Xu, Danyal Mohaddes, and Yi Wang. Llm agent for fire dynamics simulations. arXiv (Cornell University), 2024.

[29] Chenchen Ye, Ziniu Hu, Yihe Deng, and Zijie Huang. Mirai: Evaluating llm agents for event forecasting. arXiv (Cornell University), 2024.

[30] Bo Wang, Weiyi He, Shenglai Zeng, and Zhen Xiang. Unveiling privacy risks in llm agent memory. arXiv (search), 2025.

[31] Kaiyuan Zhang, Zian Su, Pin-Yu Chen, and Elisa Bertino. Llm agents should employ security principles. arXiv (search), 2025.

[32] Zichen Chen, Jiaao Chen, Jianda Chen, and Misha Sra. Standard benchmarks fail - auditing llm agents in finance must prioritize risk. arXiv (search), 2025.

[33] Autovqa-g: Self-improving agentic framework for automated visual question answering with grounding. Preprint.

[34] Rongsheng Hu, Yuan Liu, Runwei Guan, Yicheng Di, Jiayu Bao, Yi Jin, Hongjian Shi, Yuan Hong, Lin Yu, Ruhui Ma, and Rajkumar Buyya. Vqa-g annotator: A self-improving agentic framework for high-fidelity grounded dataset synthesis, 2026. Preprint.

[35] Zhuoran Xiao, Chenhui Ye, Yunbo Hu, and Honggang Yuan. 6g llm agents: A novel paradigm for task-oriented physical-layer automation. arXiv (search), 2026.

[36] Loris Gaven, Thomas Carta, Clément Romac, and Cédric Colas. Magellan: Metacognitive predictions of learning progress guide autotelic llm agents in large goal spaces. arXiv (search), 2025.

[37] Thomas Helmuth, Edward Pantridge, and Lee Spector. Lexicase selection of specialists. Preprint.

[38] Thomas Krismayer, Michael Vierhauser, Rick Rabiser, and Paul Grünbacher. Comparing constraints mined from execution logs to understand software evolution. arXiv (OAI-PMH), 2020.

[39] Yuanyuan Xu, Wenjie Zhang, Yin Chen, Xuemin Lin, and Ying Zhang. Self-evolving agents as dynamic graph transformation: A survey and new perspective, 2026. Preprint.

[40] Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. Symbolic learning enables self-evolving agents, 2026. Preprint.

[41] Association for Computational Linguistics 2026, Lin Cheong, Megha Gandhi, Soumya Smruti Mishra, Yijun Tian, Jiongxiao Wang, Yawei Wang, Zhichao Xu, Panpan Xu, and Qiaojing Yan. Reinforcement learning for self-improving agent with skill library, 2026. Preprint.

[42] Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. On the structural memory of llm agents. arXiv (Cornell University), 2024.

[43] Kostas Hatalis, Despina Christou, Joshua Myers, Steve R. Jones, Keith Lambert, Adam Amos-Binks, Zohreh A. Dannenhauer, and Dustin Dannenhauer. Memory matters: The need to improve long-term memory in llm agents. Preprint.

[44] Ang Li, Yin Zhou, Vethavikashini Chithrra Raghuram, and Tom Goldstein. Commercial llm agents are already vulnerable to simple yet dangerous attacks. arXiv (search), 2025.

[45] Marin Biloš, Bertrand Charpentier, and Stephan Günnemann. Uncertainty on asynchronous time event prediction, 2026. Preprint.

[46] Huan ang Gao, Jiayi Geng, Wenyue Hua, and Mengkang Hu. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. arXiv (search), 2026.

[47] Zhe Ren, Yimeng Chen, Dandan Guo, and Guowei Rong. Self-improvements in modern agentic systems: A survey. arXiv (search), 2026.

[48] An Lanji, Dawei Liu, Jin Li, and Haoran Xu. Diffuseagent-mi: Distributionally-grounded, tool-integrated self-evolving agents for faithful visual reasoning. arXiv (Cornell University), 2026.

[49] Xuechen Liang, Yangfan He, Yinghui Xia, and Xinyuan Song. Sage: Self-evolving agents with reflective and memory-augmented abilities. arXiv (Cornell University), 2026.

[50] Yufei He, Juncheng Liu, Yue Liu, and Yibo Li. Evotest: Evolutionary test-time learning for self-improving agents. arXiv (Cornell University), 2026.

[51] Gaodan Fang, Vatche Isahagian, K. R. Jayaram, and Ritesh Kumar. Trajectory-informed memory generation for self-improving agent systems. arXiv (Cornell University), 2026.

[52] Ye Yu, Xiaopeng Yuan, Haibo Jin, and Heming Liu. Do self-evolving agents forget? capability degradation and preservation in lifelong llm agent adaptation. arXiv (Cornell University), 2026.

[53] Xing Zhang, Yanwei Cui, Guanghui Wang, and Ziyuan Li. The blind curator: How a biased judge silently disables skill retirement in self-evolving agents. arXiv (search), 2026.

[54] Zhaohan Meng, Zaiqiao Meng, Siwei Liu, and Hao Xu. Sse-bio: A structured self-evolving agent with agentic retrieval policy for multi-hop biomedical reasoning. arXiv (Cornell University), 2026.

[55] Qizheng Zhang, Changran Hu, Shubhangi Upasani, and Boyuan Ma. Agentic context engineering: Evolving contexts for self-improving llm systems. arXiv (Cornell University), 2026.

[56] Haozhen Zhang, Quanyu Long, Jianzhu Bao, and Tao Feng. Memskill: Learning and evolving memory skills for self-evolving agents. arXiv (Cornell University), 2026.

[57] Hengyu An, Minxi Li, Naen Xu, Chunyi Zhou, Xiaogang Xu, Tianyu Du, Jinbao Li, and Shouling Ji. Permemsafe: Benchmarking implicit personalized safety of long-horizon self-evolving agents, 2026. Preprint.

[58] Hui Xue and Fan Yang. Rethinking self-evolving agents: Do we still need prescribed optimization pipelines? arXiv (Cornell University), 2026.

[59] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. Llm agents for interactive workflow provenance: Reference architecture and evaluation methodology, 2026. Preprint.

[60] Jin Han, Sining Zhoubian, Xinzheng Lu, Zhen Hu, Jun Ma, and Jia-Rui Lin. Bim design assistant driven by llm agents, 2026. Preprint.

[61] Qiming Ye, Peixain Zhang, Yupeng He, and Zifan Peng. Behind evomap: Characterizing a self-evolving agent-to-agent collaboration network. arXiv (Cornell University), 2026.

[62] Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, and Jin Xu. Odysseybench:. arXiv (search), 2026.

[63] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, and Derek Duenas. Agentharm: A benchmark for measuring harmfulness of llm agents. arXiv (Cornell University), 2026.

[64] Dianzhi Yu, Vireo Zhang, Hongru Wang, and Yanyu Chen. Dynamic mixture of latent memories for self-evolving agents. arXiv (Cornell University), 2026.

[65] Pranath Reddy. Selective memory retention for long-horizon llm agents. arXiv (search), 2026.

[66] Bingqing Wei, Zhongyu Xia, Dingai Liu, and Xiaoyu Zhou. Elite: Experiential learning and intent-aware transfer for self-improving embodied agents. arXiv (Cornell University), 2026.

[67] Sihang Jiang, Lipeng Ma, Zhonghua Hong, and Keyi Wang. Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment. arXiv (Cornell University), 2026.

[68] Hongqiang Lin, Chao Liu, Xiaofan Bai, and Xuan Jin. Beyond endpoint performance: Process-level evaluation of self-evolving agents. arXiv (Cornell University), 2026.

[69] Yifan Song, Da Yin, Xiang Yue, and Jie Huang. Trial and error: Exploration-based trajectory optimization for llm agents. arXiv (search), 2026.

[70] Yunpeng Zhai, Shuchang Tao, Cheng Chen, and Anni Zou. Agentevolver: Towards efficient self-evolving agent system. arXiv (Cornell University), 2026.

[71] Min Yang, Jinghua Piao, Xu Xia, and Xiaochong Lan. Skillmaster: Toward autonomous skill mastery in llm agents. arXiv (search), 2026.

[72] Michael Nguyen, Quoc Nguyen, and Paul Vuong. Recursive self-evolving agents via held-out selection. arXiv (Cornell University), 2026.

[73] Diandian Guo, Cong Cao, Fangfang Yuan, and Yingqi Wang. Self-authored verification is unreliable in heuristic self-improving agents. arXiv (Cornell University), 2026.

[74] Yihao Wang, Linhan Xia, Rui Liu, and Zhaofeng Zhang. Sage: A statistical acceptance gate for self-evolving agents. arXiv (Cornell University), 2026.

[75] Vansh Wahi. Llm-as-a-judge is not an oracle: Why self-improving agents need deterministic guardrails. arXiv (Cornell University), 2026.

[76] Bo Deng, Kang Zhou, Lifan Guo, and Chongyang Tao. Finevo-bench: A longitudinal benchmark. arXiv (Cornell University), 2026.

[77] Jialuo Chen, Lingqi Jiang, Xinhao Deng, and Xiaohu Du. When experience becomes instruction: Trajectory poisoning in self-evolving agent skill systems. arXiv (search), 2026.

[78] Zonghao Ying, XiangFan Wu, Huiyu Wu, and Xing Zheng. Skilljack: Persistent skill backdoors in self-evolving agents. arXiv (Cornell University), 2026.

[79] Qiyan Zhao, Xiaofeng Zhang, Bo Liu, and Minda Chen. Coalition-aware skill reliability for self-evolving agents. arXiv (Cornell University), 2026.

[80] Xing Zhang, Guanghui Wang, Yanwei Cui, and Ziyuan Li. Who verifies the verifier? co-evolving inspectable graders with self-improving agents. arXiv (Cornell University), 2026.

[81] Zayx Shawn. Pace: Anytime-valid acceptance tests for self-evolving agents. arXiv (Cornell University), 2026.

[82] Shirley Wu, Shiyu Zhao, Qian Huang, and Kexin Huang. Avatar: Optimizing llm agents for tool usage via contrastive reasoning. arXiv (search), 2026.

[83] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, and Yankai Chen. Coevoskills: Self-evolving agent skills via co-evolutionary verification. arXiv (Cornell University), 2026.

[84] Siru Ouyang, Jun Yan, Yanfei Chen, and Rujun Han. Skillos: Learning skill curation for self-evolving agents. arXiv (Cornell University), 2026.

[85] Tianxin Wei, Ting-Wei Li, Zhining Liu, and Xuying Ning. Agentic reasoning for large language models foundations · evolution · collaboration tianxin wei1†. arXiv (Cornell University), 2026.

[86] Zefeng Wang, Minxi Yan, Jinhe Bi, and Sikuan Yan. Metaskill-evolve: Recursive self-improvement of llm agents via two-timescale meta-skill evolution. arXiv (search), 2026.

[87] Xing Zhang, Guanghui Wang, Yanwei Cui, and Ziyuan Li. Who grades the grader? co-evolving evaluation metrics and skills for self-improving llm agents. arXiv (search), 2026.

[88] Dongdong Hua, Yifei Sun, Renhong Huang, and Feng Gao. Ptcg-bench: Can llm agents master pokémon trading card game? arXiv (search), 2026.